Computer-Aided Design 198 (2026) 104087

Contents lists available at ScienceDirect

Computer-Aided Design

journal homepage: www.elsevier.com/locate/cad

CADDesigner: Conceptual CAD model generation with a general-purpose agent☆

Fengxiao Fan [a,1], Jingzhe Ni [a,1], Xiaolong Yin [a], Sirui Wang [a], Xingyu Lu [a], Qiang Zou [b], Ruofeng Tong [a], Min Tang [a], Peng Du [b,*]

[a] Zhejiang University, China
[b] State Key Lab of CAD and CG, Zhejiang University, China

ARTICLE INFO

Keywords:
CAD code generation
ReAct agent
Multimodal input
Large Language Models (LLMs)

ABSTRACT

Computer-Aided Design (CAD) is widely used for conceptual design and parametric 3D modeling, but typically requires a high level of expertise from designers. To lower the entry barrier and facilitate early-stage CAD modeling, we present CADDesigner, an LLM-powered agent for conceptual CAD design. The agent accepts both textual descriptions and sketches as input, engaging in interactive dialogue with users to refine and clarify design requirements through comprehensive requirement analysis. Built upon a novel Explicit Context Imperative Paradigm (ECIP), the agent generates high-quality CAD modeling code. During the generation process, the agent incorporates iterative visual feedback to improve model quality. Generated design cases can be stored in a structured knowledge base, providing a mechanism for continual knowledge accumulation and future improvement of code generation. Experimental results show that CADDesigner achieves competitive performance and outperforms representative baselines on conceptual CAD model generation tasks.

1. Introduction

Conceptual design of Computer-Aided Design (CAD) models often begins with incomplete or abstract specifications, requiring designers to translate rough ideas into precise parametric models. Creating such 3D models using traditional CAD software typically demands substantial expertise and familiarity with complex modeling operations, which can be a barrier for novice users and slow down early-stage product development. Traditional CAD platforms – such as OnShape, AutoCAD, SolidWorks, and CATIA – primarily rely on manual sketching and extrusion workflows, making iterative prototyping time-consuming and error-prone. However, recent advances in artificial intelligence, particularly the emergence of LLMs, present promising opportunities to automate CAD model generation, thereby reducing the entry barrier and accelerating the design workflow.

Early research on automatic CAD model generation has primarily focused on parametric modeling approaches [1–4]. However, these methods are constrained by the limited diversity of training data and the representational capacity of their model outputs, supporting only a narrow set of CAD operations. Recent work leveraging LLMs for CAD code generation has shown promise in overcoming these limitations [5–7]. Fine-tuned LLMs can interpret multimodal user inputs and generate corresponding CAD modeling code. Nevertheless, fine-tuning open-source LLMs requires substantial GPU resources, and high-quality CAD training datasets remain scarce, limiting the diversity and quality of the generated code.

To address these challenges, we propose CADDesigner, an LLM-powered agent for conceptual CAD design. CADDesigner accepts both textual descriptions and sketches as input and engages in interactive dialogue with users to refine design requirements. The system combines knowledge-constrained code generation with iterative visual feedback, enabling the agent to generate high-quality CAD modeling code that better aligns with user design intent.

To further improve code generation quality, we propose the Explicit Context Imperative Paradigm (ECIP), an LLM-oriented representation style implemented through a lightweight API layer built on top of CadQuery [8]. ECIP explicitly represents modeling context and restructures CAD operations into an explicit imperative workflow, enabling clearer state transitions, more reliable code generation, and easier iterative correction while remaining aligned with CadQuery semantics.

☆ This article is part of a Special issue entitled: 'AI+Design' published in Computer-Aided Design.

* Corresponding author.
E-mail addresses: fxfan@zju.edu.cn (F. Fan), nijingzhe@zju.edu.cn (J. Ni), yinxiaoloong@zju.edu.cn (X. Yin), siruiw@zju.edu.cn (S. Wang), xingyulu@zju.edu.cn (X. Lu), qiangzou@zju.edu.cn (Q. Zou), trf@zju.edu.cn (R. Tong), tang_m@zju.edu.cn (M. Tang), dp@zju.edu.cn (P. Du).

[1] Equal Contributions.

https://doi.org/10.1016/j.cad.2026.104087
Received 12 December 2025; Received in revised form 4 April 2026; Accepted 12 May 2026
Available online 23 May 2026

计算机辅助设计 198 (2026) 104087

目录列表可访问ScienceDirect

计算机辅助设计

期刊首页：www.elsevier.com/locate/cad

CAD设计师：使用通用智能体生成概念性CAD模型☆

范凤晓[a,1]，倪静哲[a,1]，尹晓龙[a]，王思瑞[a]，卢兴宇[a]，邹强[b]
罗峰通[a]、唐敏[a]、杜鹏[b,*]

[a] 中国浙江大学
[b] 中国浙江大学CAD与CG国家重点实验室

ARTICLE INFO

关键词:
CAD代码生成
ReAct代理程序
多模态输入
大型语言模型（LLM）

ABSTRACT

计算机辅助设计（CAD）广泛应用于概念设计和参数化三维建模领域，但通常需要设计师具备较高的专业素养。为降低入门门槛并促进早期阶段的CAD建模工作，我们推出了CADDesigner——一款基于大语言模型（LLM）驱动的概念CAD设计智能代理。该智能代理可接收文本描述和草图作为输入，通过与用户进行交互式对话，并结合全面的需求分析来完善和明确设计要求。该系统基于创新的显式上下文指令范式（ECIP）构建，能够生成高质量的CAD建模代码；在生成过程中，智能代理会实时整合迭代视觉反馈以提升模型质量。生成的设计案例可存储于结构化知识库中，从而实现知识的持续积累及代码生成功能的持续优化。实验结果表明，在概念CAD模型生成任务中，CADDesigner展现出优异性能，其表现远超主流基准方法。

1. 引言

计算机辅助设计（CAD）模型的概念设计通常始于不完整或抽象的规格描述，要求设计师将初步构想转化为精确的参数化模型。使用传统CAD软件创建此类三维模型通常需要深厚的专业知识以及对复杂建模操作的熟练掌握，这可能成为新手用户的障碍，并拖慢产品开发初期进度。传统的CAD平台（如OnShape、AutoCAD、SolidWorks和 CATIA）主要依赖手工绘图与挤出加工流程，导致迭代原型制作既耗时又易出错。然而，人工智能领域的最新进展——尤其是大型语言模型（LLM）的出现——为自动化生成CAD模型提供了广阔前景，从而降低入门门槛并加速设计流程。

早期关于自动CAD模型生成的研究主要集中在参数化建模方法[1–4]上。然而，这些方法受到训练数据多样性的限制以及模型输出表示能力的制约，仅能支持有限范围的CAD操作。近期利用大语言模型（LLM）进行CAD代码生成的研究在克服这些局限方面展现出潜力[5–7]：微调后的LLM能够解析多模态用户输入并生成相应的CAD建模代码。但微调开源LLM需要大量GPU资源，且高质量的CAD训练数据集仍十分匮乏，这限制了生成代码的多样性和质量。

为应对这些挑战，我们推出了CADDesigner——一款基于大语言模型（LLM）驱动的概念性CAD设计智能代理。该系统可同时接收文本描述和草图作为输入，并通过交互对话与用户共同完善设计需求。它将知识约束型代码生成与迭代式视觉反馈相结合，使智能代理能够生成与用户设计意图高度契合的高质量CAD建模代码。

为进一步提升代码生成质量，我们提出了显式上下文命令范式（ECIP）——这是一种面向大型语言模型的表示方式，通过基于CadQuery[8]构建的轻量级API层实现。 ECIP 能够明确呈现建模上下文，并将CAD操作重构为显式的命令式工作流程，从而实现更清晰的状态转换、更可靠的代码生成以及更便捷的迭代修正，同时始终符合CadQuery的语义规范。图1展示了CADDesigner生成的代表性结果，我们的主要贡

☆ 本文为《计算机辅助设计》期刊发表的特刊《AI+设计》系列文章之一。

* 通讯作者。
电子邮件地址: fxfan@zju.edu.cn（范飞），nijingzhe@zju.edu.cn（倪静），yinxiaoloong@zju.edu.cn（尹欣霞），siruiw@zju.edu.cn（王思瑞），xingyulu@zju.edu.cn（卢欣），qiangzou@zju.edu.cn（邹强），trf@zju.edu.cn（童瑞），tang_m@zju.edu.cn（唐敏），dp@zju.edu.cn（杜鹏）。

[1] 平等贡献。

https://doi.org/10.1016/j.cad.2026.104087
接收日期：2025年12月12日；修订版接收日期：2026年4月4日；接受日期：2026年5月12日；在线发布日期：2026年5月23日

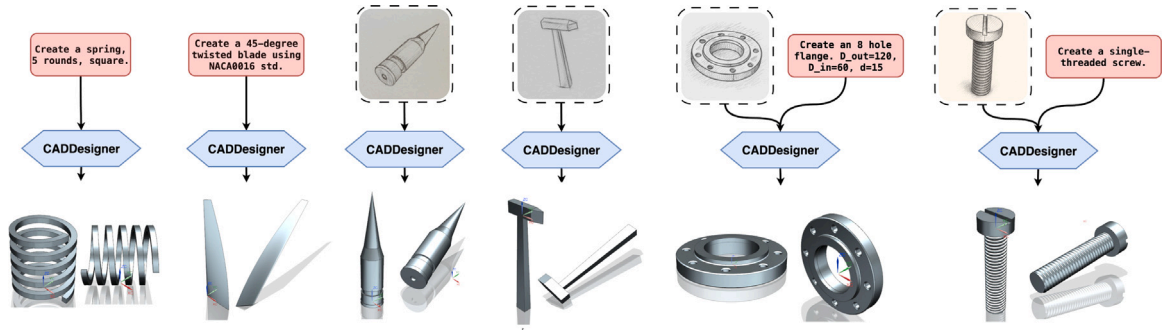

Fig. 1. Demonstration of various CAD models generated by CADDesigner. Our method supports multimodal input and a broad range of CAD operations, including extrusion, revolution, fillet/chamfer, sweeping, lofting, etc., as well as the creation of standard components such as flanges and screws.

Representative results generated by CADDesigner are shown in Fig. 1, and our main contributions are summarized as follows:

- We present CADDesigner, a framework for conceptual CAD model generation. CADDesigner accepts textual descriptions and sketches as user input, and integrates tools for requirement analysis, knowledge-constrained code generation, and vision-based error correction to generate CAD scripts that satisfy user intent.
- We introduce an explicit context imperative paradigm for CAD modeling script generation based on an imperative function-calling structure. This paradigm decouples individual CAD operations and improves code generation quality through explicit type annotations, LLM-friendly error representations, and self-evolving capabilities. It supports a wide range of operations, including extrusion, revolution, fillet, chamfer, sweeping, lofting, etc.

The remainder of this paper is organized as follows. We begin with a review of related work. We then introduce our proposed CAD modeling code generation framework, CADDesigner, along with the novel code generation paradigm designed to enhance code quality. Next, we describe our experimental setup and present the results. Finally, we provide a comprehensive analysis and comparison to highlight the strengths and limitations of the proposed approach.

2. Related work

This section provides a comprehensive review of four key areas relevant to our work: parametric CAD modeling, LLM-driven CAD generation, agent-based model generation, and parametric CAD modeling SDKs.

2.1. Parametric CAD model generation

Parametric CAD model generation approaches commonly employ supervised learning techniques to produce sequences of CAD commands, which can be imported into CAD modeling software to generate editable, parametric files [1,9]. Systems such as DeepCAD [1], Fusion 360 Gallery [9], SkexGen [10], and Diffusion-CAD [11] adopt customized neural network architectures trained to generate command sequences. However, they primarily support only basic modeling operations such as sketching and extrusion, and struggle to generalize to more complex modeling procedures.

Additional efforts [3,12,13] explore CAD model reconstruction from point cloud data, aiming to recover structured geometry from unorganized 3D observations. More recent methods [14,15] further attempt to infer structured, editable CAD models directly from unstructured RGB images (either single- or multi-view), bridging the gap between 2D visual understanding and 3D CAD modeling. Despite these advancements, existing approaches remain limited in their ability to generate complex CAD models, primarily due to constrained training data diversity and limited output representation capacity.

2.2. LLM-based CAD model generation

Fine-tuned LLMs and vision-language models (VLMs) have emerged as a prominent direction for CAD model generation, enabling the synthesis of CAD command sequences or Python code from multimodal input. Cad-LLM [16] and CadVLM [17] utilize fine-tuned LLMs to generate parametric models from sketches and images. CAD-MLLM [18] targets multimodal inputs – including text, images, and point clouds – for parametric model generation. CAD-Llama [5] proposes a hierarchical annotation pipeline that transforms command sequences into semantically structured CAD code. This is followed by adaptive pre-training and instruction tuning, enabling LLMs to generate high-fidelity parametric 3D models. CAD-Recode [7] converts point clouds into sequential representations, then uses a pre-trained LLM (Qwen2-1.5B [19]) to generate corresponding CAD code. Text-to-CadQuery [20] directly maps text descriptions to CadQuery code, leveraging pre-trained LLMs' capabilities in Python generation and spatial reasoning.

While these methods demonstrate strong generation capabilities, they remain limited by several factors: the computational cost of fine-tuning large models, the closed nature of many commercial LLMs, and the scarcity of high-quality, diverse training datasets for CAD.

2.3. Agent-based CAD model generation

Another research direction focuses on constructing CAD generation agents using prompt engineering and ultra-large foundation models, circumventing the need for additional training. CAD-Assistant [21] presents a general-purpose CAD agent framework that integrates visual and language models (VLLMs) as planners to generate Python code for FreeCAD [22], achieving zero-shot capability across diverse CAD tasks. SeekCAD [23] combines retrieval-augmented generation (RAG), visual feedback, and a chain-of-thought (CoT) mechanism to generate customized CAD modeling code with self-optimization. 3D-PreMise [24] investigates the potential and limitations of LLMs in program synthesis for manipulating 3D software and generating parametrically controlled shapes. CADCodeVerify [25] incorporates iterative validation and refinement loops, using visual language models to pose and answer verification questions about the generated CAD outputs.

In contrast to these works, CADDesigner accepts multimodal user inputs – text and sketches – to capture detailed design requirements, engages in interactive refinement with users, and enhances code quality and model fidelity through visual feedback and iterative optimization.

2.4. Parametric CAD modeling SDKs

Beyond modeling methods, existing CAD modeling SDKs play a crucial role as the underlying representations for code-based CAD generation. Most Python-based parametric CAD frameworks are built upon OpenCASCADE Technology (OCCT), a widely used boundary representation modeling kernel. CadQuery provides a high-level Python

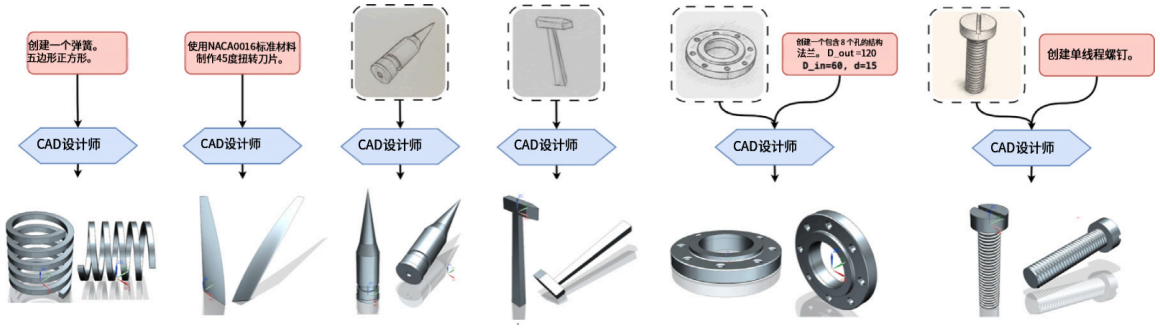

图 1.CADDesigner 生成的多种 CAD 模型示例。我们的方法支持多模态输入及广泛的 CAD 操作功能，包括拉伸、旋转、倒角/修边、扫描、曲面拉削等，并可创建法兰、螺钉等标准部件。

献总结如下：

- 我们推出CADDesigner，这是一个用于生成概念性CAD模型的框架。该框架接受文本描述和草图作为用户输入，并整合了需求分析、知识约束代码生成以及基于视觉的错误校正等工具，以生成符合用户需求的CAD脚本。
- 我们提出了一种基于命令式函数调用结构的显式上下文命令范式，用于生成CAD建模脚本。该范式实现了各CAD操作之间的解耦，并通过显式的类型注释、符合大语言模型要求的错误表示机制以及自适应优化能力提升了代码生成质量。它支持包括拉伸、旋转、倒圆、倒角、扫描、曲面修整等多种操作类型。

本文其余部分结构安排如下：首先回顾相关研究进展；随后介绍我们提出的CAD建模代码生成框架CADDesigner以及旨在提升代码质量的创新代码生成范式；接着阐述实验设计并呈现研究成果；最后通过全面分析与对比，突出该方法的优势与局限性。

2. 相关工作

本节对与我们工作相关的四个关键领域进行了全面综述：参数化CAD建模、基于LLM的CAD生成、基于智能体的模型生成以及参数化CAD建模SDK。

2.1. 参数化CAD模型生成

参数化CAD模型生成方法通常采用监督学习技术来生成一系列CAD命令序列，这些序列可导入CAD建模软件以生成可编辑的参数化文件[1,9]。诸如DeepCAD[1]、Fusion 360 Gallery[9]、SkexGen[10]以及Diffusion-CAD[11]等系统均采用了经过训练的定制化神经网络架构来生成命令序列。然而，这些系统主要仅支持草图绘制和拉伸等基础建模操作，并难以扩展到更复杂的建模流程。

已有研究[3,12,13]进一步探索了基于点云数据重建CAD模型的方法，旨在从无序的三维观测数据中还原结构化几何形态。更近期的方法[14,15]则尝试直接从非结构化的RGB图像（单视图或多视图）中推断出可编辑的结构化CAD模型，从而弥合二维视觉理解与三维CAD建模之间的差距。尽管取得了这些进展，现有方法在生成复杂CAD模型方面仍存在局限性，主要源于训练数据多样性的不足以及输出表征能力有限。

2.2. 基于LLM的CAD模型生成

微调后的大语言模型（LLM）和视觉语言模型（VLM）已成为CAD模型生成领域的主流方向，能够从多模态输入中合成CAD命令序列或Python代码。Cad-LLM[16]与CadVLM[17]利用微调后的LLM从草图和图像中生成参数化模型；CAD- MLLM[18]则针对文本、图像及点云等多模态输入进行参数化模型生成；CAD-Llama[5]提出了一种分层标注流程，可将命令序列转化为语义结构化的CAD代码，并通过自适应预训练与指令调优使LLM生成高保真度的参数化3D模型；CAD-Recode[7]先将点云转换为序列化表示，再利用预训练LLM（Qwen2-1.5B[19]）生成对应CAD代码；Text-to-CadQuery[20]则直接将文本描述映射为CadQuery代码，充分发挥预训练LLM在Python代码生成与空间推理方面的优势。

尽管这些方法展现出强大的生成能力，但仍受到若干因素的限制：大型模型微调所需的计算成本、许多商业大语言模型的封闭性，以及面向计算机辅助设计（CAD）领域高质量、多样化训练数据集的匮乏。

2.3. 基于代理的CAD模型生成

agent

另一研究方向聚焦于利用提示工程和超大规模基础模型构建CAD生成代理，从而避免额外训练需求。[21]中的CAD-Assistant提出了一种通用CAD代理框架，该框架将视觉与语言模型（VLLMs）作为规划器整合使用，为FreeCAD[22]生成Python代码，实现了跨多种CAD任务的零样本学习能力。SeekCAD[23]结合检索增强生成（RAG）、视觉反馈及思维链（CoT）机制，通过自优化功能生成定制化CAD建模代码。3D-PreMise[24]探讨了大型语言模型在三维软件操作编程合成及参数化形状生成方面的潜力与局限性。CADCodeVerify[25]采用迭代验证与优化循环机制，利用视觉语言模型对生成的CAD输出进行验证问题的提出与解答。

与这些作品不同，CADDesigner支持多模态用户输入（文本和草图）以捕捉详细的设计需求，与用户进行交互式优化，并通过视觉反馈和迭代优化提升代码质量与模型保真度。

2.4. 参数化CAD建模SDK

除建模方法外，现有的CAD建模SDK作为基于代码的CAD生成的基础表示工具发挥着关键作用。大多数基于Python的参数化CAD框架均基于OpenCASCADE Technology（OCCT）构建——这是一种广泛使用的边界表示建模核心库。CadQuery则提供了一个高级Python接口层。

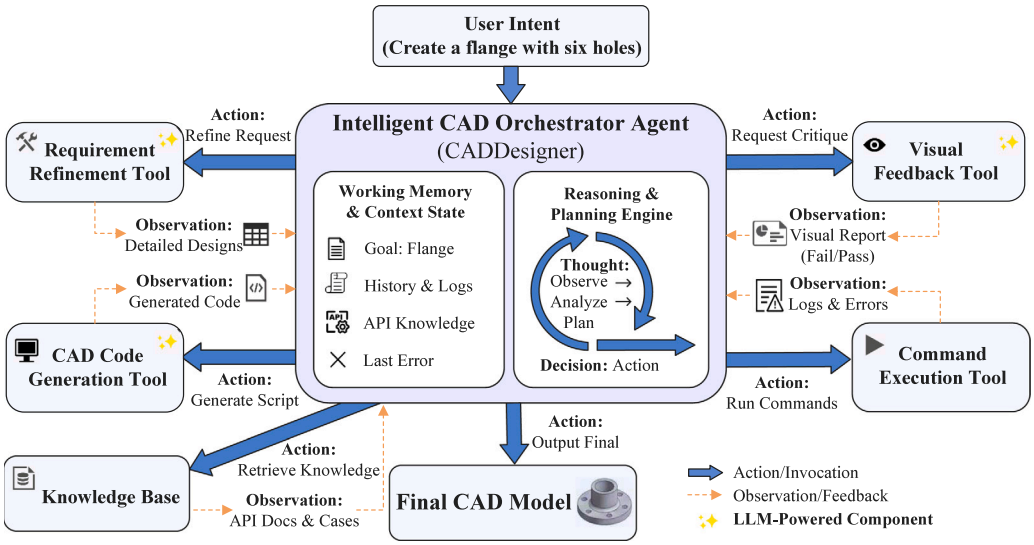

Fig. 2. The Intelligent CAD Orchestrator Agent, CADDesigner, follows a ReAct-style paradigm to progressively transform user requirements into valid CAD models through iterative reasoning, tool execution, and feedback refinement. It first refines user requirements into detailed designs, generates executable modeling code using domain APIs, and analyzes execution results via both symbolic (e.g., shell logs and errors) and visual feedback (e.g., rendered 3D views).

interface with a Fluent API based on method chaining and workplane abstractions [8], enabling concise modeling workflows through implicit context propagation. Build123d adopts a different design based on context managers (e.g., `with` statements) and operator-style modeling by overloading operators in Python, offering tighter integration with native Python constructs and improved modularity [26].

However, these SDKs are not specifically designed for LLM-based code generation. CadQuery's Fluent API relies on implicit internal state transitions, which can obscure intermediate modeling context and increase the difficulty for LLMs to track dependencies across operations. Despite its explicit context declaration, build123d's reliance on symbolic operators (@, +, −) for complex modeling may make semantic intent less transparent for LLM-based code generation, especially compared with explicit function-call interfaces. Such implicit context handling and lack of well-defined semantics can lead to ambiguity and reduced reliability in generated code. These limitations motivate the need for a more explicit and LLM-friendly CAD representation, which we address through the proposed Explicit Context Imperative Paradigm (ECIP).

3. CAD modeling architecture

In this section, we introduce the CADDesigner architecture, which features a ReAct-style agent interacting with a diverse set of integrated CAD tools to iteratively generate and refine CAD models. The overall framework is illustrated in Fig. 2.

3.1. ReAct agent rather than workflow

CADDesigner employs a ReAct-style [27] agent-centric architecture, where a central agent governs the entire modeling loop through iterative reasoning, tool execution, and feedback integration.

Unlike traditional hard-coded workflows, our architecture leverages prompt-based workflow specifications, where the system prompts encode procedural guidance for the agent. This design allows for flexible and revisable task orchestration, as the workflow is not statically programmed but interpreted at runtime by the agent. As a result, users can intervene at any stage of the process, like adding constraints, correcting directions, or adjusting goals, leading to a truly interruptible and human-in-the-loop modeling cycle.

The ReAct-style loop includes reasoning, acting, and reflecting: the agent expands coarse inputs into parameterized specs, generates executable code, renders outputs, evaluates results, and iteratively refines the model until user intent is satisfied. By consolidating control within the agent, we enable adaptive tool use and interpretable, interactive CAD automation in under-specified or evolving design tasks.

3.2. CAD tools

The intelligent orchestration relies on a diverse suite of CAD-oriented tools. Each tool addresses a specific stage within the design-to-modeling pipeline and works in synergy under the direction of the ReAct agent. This subsection describes the CAD tools integrated in our pipeline.

3.2.1. Core CAD toolset

CADDesigner employs various CAD-specific tools to support its design-to-modeling pipeline. Central to this set are four tools, denoted $\mathcal{T} = \{T_1, T_2, T_3, T_4\}$, each fulfilling a distinct role.

- **Requirement Refinement Tool** $T_1 : \mathcal{I} \rightarrow \mathcal{D}$ maps user-provided multimodal input $\mathcal{I}$ to detailed design descriptions $\mathcal{D}$, refining initial requirements into structured design specifications.
- **Code Generation Tool** $T_2 : \mathcal{D} \rightarrow \mathcal{C}$ translates finalized design descriptions into executable CAD modeling code $\mathcal{C}$. A new code paradigm for CAD code generation has been designed and used by T_2. This is discussed in detail in Section 4.
- **Command Execution Tool** $T_3 : \mathcal{C} \rightarrow \mathcal{M}$ executes the generated Python code $\mathcal{C}$ to produce the corresponding CAD model $\mathcal{M}$ by running shell commands.
- **Visual Feedback Tool** $T_4 : \{\mathcal{M}, D\} \rightarrow (P, F)$ outputs both a high-level termination signal $P \in \{0, 1\}$, where $P = 1$ triggers termination, and structured diagnostic feedback F for refinement. Inside this tool, a multi-view rendering result of model $\mathcal{M}$ will be generated. We denote this render result as $\mathcal{V}$. Thus T_4 can be separated into 2 steps: $T_4^1 : \mathcal{M} \rightarrow \mathcal{V}$, and $T_4^2 : \{\mathcal{V}, D\} \rightarrow (P, F)$

The sequential composition of these core tools forms the main functional pipeline:

$$T_4^1 \circ T_3 \circ T_2 \circ T_1 : \mathcal{I} \rightarrow \mathcal{V}. \tag{1}$$

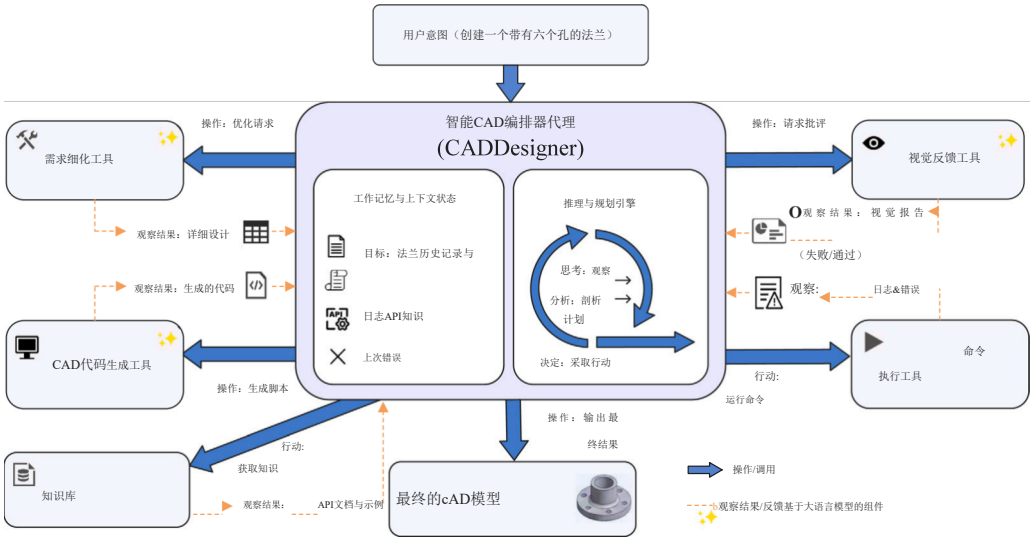

图2.智能CAD编排代理CADDesigner遵循ReAct风格范式，通过迭代推理、工具执行及反馈优化，逐步将用户需求转化为有效的CAD模型。该系统首先将用户需求细化为详细设计方案，利用领域API生成可执行建模代码，并通过符号化反馈（如外壳日志与错误信息）和可视化反馈（如渲染的3D视图）对执行结果进行分析。

该接口基于方法链和工作平面抽象概念的Fluent API[8]，通过隐式上下文传播实现简洁的建模工作流程。Build123d则采用不同的设计架构：基于上下文管理器（如语句形式）以及通过重载Python运算符实现的运算符式建模，从而与Python原生构造函数实现更紧密集成，并提升模块化程度[26]。

然而，这些SDK并非专为基于大语言模型（LLM）的代码生成而设计。CadQuery的Fluent API依赖隐式的内部状态转换机制，这可能会掩盖中间建模过程中的上下文信息，增加LLM追踪操作间依赖关系的难度。尽管build123d采用了显式上下文声明，但其在复杂建模中过度依赖符号运算符（@、+、−），可能导致基于LLM的代码生成难以清晰呈现语义意图——尤其与显式函数调用接口相比更为明显。这种隐式上下文处理方式以及缺乏明确定义的语义规范，容易导致代码生成结果存在歧义且可靠性降低。这些局限性凸显了开发更明确、更适配LLM的CAD表示方法的必要性，我们通过提出的显式上下文指令范式（ECIP）来解决这一问题。

3. CAD建模架构

在本节中，我们介绍CADDesigner架构，该架构采用ReAct风格的智能体与多种集成化的CAD工具进行交互，以迭代方式生成并优化CAD模型。整体框架如图2所示。

3.1. ReAct代理而非工作流

CADDesigner采用ReAct风格的[27]代理中心架构，其中中央代理通过迭代推理、工具执行及反馈整合来调控整个建模流程。

与传统的硬编码工作流不同，我们的架构采用基于提示的工作流规范设计——系统生成的提示语句为智能体提供了操作流程指引。这种设计实现了灵活且可调整的任务调度：工作流并非静态编程，而是在运行时由智能体动态解析执行。因此用户可在流程的任何阶段进行干预（如添加约束条件、修正操作指令或调整目标），从而形成真正可中断且具备人机交互机制的建模循环。

ReAct风格的循环流程包含推理、执行与反思三个环节：智能体将粗略输入转化为参数化配置，生成可执行代码，输出结果并进行评估，随后迭代优化模型直至满足用户需求。通过将控制权集中于智能体内部，我们能够在需求不明确或不断变化的设计任务中实现自适应工具使用以及可解释、交互式的CAD自动化操作。

3.2. CAD工具

智能编排依赖于一系列多样化的面向CAD的工具。每种工具均针对设计-建模流程中的特定阶段，并在ReAct代理的指导下协同工作。本小节阐述了集成于我们流程中的CAD工具。

3.2.1. 核心CAD工具集

CAD设计师运用多种专为CAD设计打造的工具来支持其从设计到建模的全流程。其中核心包含四款工具（记作 $\tau = \{T_1, T_2, T_3, T_4\}$），每款工具均承担独特功能。

- **需求细化工具**T_1：1→D将用户提供的多模态输入1映射至详细设计描述D，将初始需求细化为结构化设计规范。
- **代码生成工具**T_2：D → C 将最终设计描述转换为可执行的CAD建模代码C。T_2设计并采用了全新的CAD代码生成代码范式，相关内容将在第4节详细讨论。
- **命令执行工具**T_3：C → M通过运行Shell命令执行生成的Python代码C，从而生成相应的CAD模型M。
- **视觉反馈工具**T_4：{M，D}→(*P*，*F*) 会同时输出高层终止信号$P \in \{0,1\}$（其中*P*=1表示终止）以及用于优化的结构化诊断反馈*F*。该工具内部将生成模型M的多视图渲染结果，我们将此渲染结果记为v。因此T_4可分解为两个步骤：$T_4^{\,1}$：M→v，以及*T24*：{v，D}→(*P*，*F*)

这些核心工具的顺序组合构成了主要的功能流程：

$$T_4^1 \circ T_3 \circ T_2 \circ T_1 : \mathcal{I} \to \mathcal{V}. \tag{1}$$

This pipeline is augmented with a closed-loop correction mechanism: after generating visual feedback $\mathcal{V}$, the system invokes T_4^2 to obtain (P, F). If $P = 0$, the feedback F is used to guide revised design generation via $T_3 \circ T_2 \circ T_1$. If $P = 1$, the process terminates successfully. Otherwise, the loop continues until T_4^2 returns a termination signal.

3.2.2. Auxiliary toolset

In addition to the core CAD toolset, CADDesigner integrates several auxiliary tools that support efficient context management and system interaction. These include:

- **SketchPad Tool**: A flexible key–value storage tool designed to mitigate the context explosion problem in LLM-based agents. SketchPad allows the agent to store arbitrary intermediate data at any time during the design process, including image paths, reference code snippets, execution results, and more. By storing context data as keyed entries instead of embedding entire data repeatedly, it significantly reduces LLM context size. SketchPad also supports automatic summarization, tag-based search, and automatic expiration of outdated entries, enabling flexible and scalable context management.
- **File Operation Tools**: These tools provide fundamental file-reading and file-writing capabilities, allowing the agent to store or load files as required throughout the CAD design pipeline. Similar to SketchPad, file content or references are stored and accessed efficiently to avoid large context overheads.

Together, these auxiliary tools complement the core CAD toolset by enabling robust, efficient, and scalable operation of the CADDesigner agent in complex multi-turn interactions.

3.3. Requirements analysis

During the conceptual design phase, designers typically have only rough descriptions or sketches, making it challenging to precisely define detailed model parameters as well as the modeling process. To address this, we support designers by conducting requirement analysis to clarify and refine their design intention.

Let the set of initial information provided by the user be $\mathcal{I} \subseteq \mathcal{I}_{\text{txt}} \times \mathcal{I}_{\text{img}}$, where $\mathcal{I}_{\text{txt}}$ denotes the set of possible text inputs (including empty text $\emptyset$) and $\mathcal{I}_{\text{img}}$ denotes the set of possible image inputs (including a null image $\emptyset$). Let $\mathcal{P}$ be a set of parameters sufficient for generating a CAD model for the current design task. The process by which CADDesigner conducts detailed design can be expressed as $\mathcal{F}_{\text{design}} : \mathcal{I} \to \mathcal{D}_{\text{detail}}$, where $\mathcal{D}_{\text{detail}}$ denotes the detailed design containing specific parameters, satisfying $\mathcal{P} \subseteq \mathcal{D}_{\text{detail}}$, and this process is accomplished by T_1. At this stage, CADDesigner enables users from different disciplines to realize their design intentions through interaction. The user's revision process for the detailed design is modeled as an iterative function:

$$R : \mathcal{D}_{\text{detail}} \times \mathcal{U} \to \mathcal{D}_{\text{detail}}, \tag{2}$$

where $\mathcal{U}$ is the set of user revision operations. After n iterations of revision, we obtain $\mathcal{D}_{\text{final}} = R^n(\mathcal{D}_{\text{detail}}, \mathcal{U})$, which is then passed to the next stage.

3.4. Knowledge constrained code generation

$\mathcal{D}_{\text{final}}$ is the detailed design confirmed by the user. The process by which the code generation tool T_2 receives $\mathcal{D}_{\text{final}}$ and generates CAD modeling code is expressed as $G : \mathcal{D}_{\text{final}} \to \mathcal{C}$. A structured knowledge base $\mathcal{K}$ is constructed to support code generation. $\mathcal{K}$ includes a set of function annotations $Anno \subseteq \mathcal{K}$ and a set of basic model cases $Case \subseteq \mathcal{K}$, which provide semantic and structural references for the code generation tool. With the assistance of the knowledge base, the code generation process is extended to:

$$G' : \mathcal{D}_{\text{final}} \times \mathcal{K} \to \mathcal{C}', \tag{3}$$

where C' denotes the code generated by referencing the knowledge base, which is semantically and structurally richer compared to the code C generated without using the knowledge base. The execution result of the code, including detailed geometry metadata (see Section 4.2.1, Metadata & Tagging) such as volumes and measurements of some parts of the model, is returned after execution. If the execution succeeds, the code can run successfully and generate a model $\mathcal{M} = T_3(C')$, which is then passed to the next stage; if the execution fails, structured error information is returned to guide the next iteration of code generation G' to produce new code. This feedback loop helps to iteratively correct the code and improve the success rate. Therefore, the knowledge base provides essential semantic and structural support for code generation, and the iterative process combined with execution feedback further enhances the likelihood of successful code generation. In addition, execution-time geometric metadata and queryable CAD-native representations provide structured signals for dimension-sensitive and constraint-related verification beyond visual inspection.

3.5. Vision-based iterative error correction

Multi-view snapshots of the generated CAD model $\mathcal{M}$ are taken by T_4^1, producing an image set $\mathcal{V} = \{\text{Img}_1, \text{Img}_2, \ldots, \text{Img}_6\}$ (corresponding to the Front-Top-Left, Back-Bottom-Right, Back-Top-Left, Front-Bottom-Right, Top, and Right views respectively). CADDesigner evaluates whether the model meets the user's intent $\mathcal{D}_{\text{final}}$ by using both a high-level termination signal $P \in \{0, 1\}$ and structured diagnostic feedback generated from multi-view observations.

Internally, P is determined by T_4^2, which involves two components:

- **Visual Question Generation** $F_{\text{vq}} : \mathcal{V} \times \mathcal{D}_{\text{final}} \to \mathcal{Q}$, which produces a set of targeted visual questions $\mathcal{Q} = \{q_1, q_2, \ldots, q_n\}$ based on the user's requirements and the multi-view images $\mathcal{V}$.
- **Visual Feedback Generation** $F_{\text{vf}} : \mathcal{V} \times \mathcal{Q} \to (P, F)$, which analyzes the multi-view images $\mathcal{V}$ to answer the questions in $\mathcal{Q}$, and outputs explicit visual feedback F along with the binary decision P. When $P = 0$, it returns visual feedback F to the code generation stage to regenerate the code; when $P = 1$, the result is delivered to the user for final judgment. The user's satisfaction with the modeling result is determined by $Sat : \mathcal{M} \to \{0, 1\}$. If $Sat(\mathcal{M}) = 1$, $\mathcal{M}$ is added to the case library $\mathcal{K}$, that is, $Case' = Case \cup \{\mathcal{M}\}$; if $Sat(\mathcal{M}) = 0$, it re-enters the code generation stage.

In the entire process, except for the user feedback links (R and Sat), all state transitions $Trans : State \to State'$ are determined independently by CADDesigner.

4. CAD code generation paradigm

In this section, we provide a detailed description of a new CAD code generation paradigm from four aspects: the Explicit Context Imperative Paradigm (ECIP), its practical realization via an ECIP-based CAD API, LLM-friendly CAD code design, and the construction of a reusable modeling knowledge base.

4.1. Explicit context imperative paradigm

In recent years, many works have employed **CadQuery** (CQ) code as an intermediate representation for CAD model generation. CadQuery commonly employs a Fluent API design paradigm [8], where operations are expressed via chained method calls to enable a highly readable and compact modeling workflow. This style relies on an *implicit internal context* C that tracks the current modeling state and is automatically passed between operations. Formally, each operation f_k updates the hidden state C_k with parameters p_k:

$$C_{k+1} = f_k(C_k, p_k). \tag{4}$$

该流程通过闭环校正机制得到增强：在生成视觉反馈V后，系统调用*T24*以获取（*P*；*F*）。若*P*=0，则利用反馈*F*通过$T_3 \circ T_2 \circ T_1$指导修订设计的生成；若*P*=1，则流程成功终止；否则循环持续执行直至*T24*返回终止信号。

3.2.2. 辅助工具集

除核心CAD工具集外，CADDesigner还集成了一系列辅助工具，以支持高效的上下文管理和系统交互。这些工具包括：

- **SketchPad工具**：一款灵活的键值存储解决方案，专为缓解基于大语言模型（LLM）的智能体中上下文数据激增问题而设计。SketchPad允许智能体在设计过程中的任何阶段存储任意中间数据，包括图像路径、参考代码片段、执行结果等。通过将上下文数据以键值形式存储而非重复嵌入完整数据，该工具显著降低了LLM模型的上下文存储规模。此外，SketchPad还支持自动摘要生成、基于标签的搜索以及过期条目的自动清除功能，实现灵活且可扩展的上下文管理。
- **文件操作工具**：这些工具提供基本的文件读取与写入功能，使代理程序能够在整个CAD设计流程中按需存储或加载文件。与SketchPad类似，文件内容及引用均可高效存储和访问，从而避免产生较大的上下文开销。

这些辅助工具共同构成了对核心CAD工具集的补充，能够确保CADDesigner代理在复杂的多轮交互过程中实现稳健、高效且可扩展的运行。

3.3. 需求分析

在概念设计阶段，设计师通常仅拥有粗略描述或草图，因此难以精确界定详细的模型参数及建模流程。为解决这一问题，我们通过开展需求分析来协助设计师明确并完善其设计意图。

设用户提供的初始信息集合为 I $\mathcal{I}_{\text{txt}} \times \mathcal{I}_{\text{img}}$，其中$\mathcal{I}_{\text{txt}}$表示可能的文本输入集合（包含空文本），$\mathcal{I}_{\text{img}}$表示可能的图像输入集合（包含空图像）。设 P 为生成当前设计任务所需 CAD 模型的参数集。CADDesigner 进行详细设计的过程可表示为$\mathcal{F}_{\text{design}}$ I→ $\mathcal{D}_{\text{detail}}$，其中 D_detail 表示包含特定参数且满足 P $\mathcal{D}_{\text{detail}}$的详细设计，该过程由$T_1$完成。在此阶段，CADDesigner 通过交互方式帮助不同领域的用户实现其设计意图。用户对详细设计的修改过程可建模为迭代函数：

$$R : \mathcal{D}_{\text{细节}} \times \mathcal{U} \rightarrow \text{Ddetail}; \tag{2}$$

其中 U 表示用户修订操作集合。经过*n*次修订迭代后，我们得到 D $_{\text{final}}$= R^n(D detail； U)，随后将其传递至下一阶段。

3.4. 基于知识的约束性代码生成

$\mathcal{D}_{\text{final}}$是经用户确认的详细设计。代码生成工具$T_2$接收$\mathcal{D}_{\text{final}}$并生成CAD建模代码的过程可表示为*G* $\mathcal{D}_{\text{final}}$ → C。为支持代码生成，构建了结构化知识库 K，其中包含一组功能注释*Anno* K 和一组基本模型案例*Case* K，这些内容为代码生成工具提供了语义和结构参考。借助该知识库，代码生成过程可扩展为：

$$G' : \mathcal{D}_{\text{最终}} \times \text{K} \rightarrow \text{C}'; \tag{3}$$

其中C′表示通过引用知识库生成的代码，与未使用知识库生成的代码C相比，在语义和结构层面都更为丰富。代码执行完成后会返回其执行结果，包括详细的几何元数据（参见第4.2.1节“元数据与标注”），例如模型某些部分的体积和尺寸参数。若执行成功，代码可正常运行并生成模型M =T_3(C′)，随后传递至下一阶段；若执行失败，则返回结构化错误信息以指导后续代码生成迭代*G′*以生成新代码。这种反馈循环有助于迭代修正代码并提升成功率。因此，知识库为代码生成提供了关键的语义与结构支持，而迭代过程结合执行反馈进一步提高了代码生成成功的可能性。此外，执行时几何元数据及可查询的CAD原生表示形式为超越视觉检查的维度敏感性验证与约束相关验证提供了结构化信息。

3.5. 基于视觉的迭代误差校正

生成的CAD模型M的多视图快照由T_4 [1]采集，形成图像集V = {Img_1, $\text{Img}_2, \ldots, \text{Img}_6$}（分别对应前-上-左、后-下-右、后-上-左、前-下-右、顶部和右侧视图）。CADDesigner通过结合高级终止信号*P* {0； 1}以及基于多视图观测生成的结构化诊断反馈，评估模型是否满足用户的最终意图$\mathcal{D}_{\text{final}}$。

在内部，*P*由*T24*决定，该值包含两个组成部分：

- **视觉问题生成**F_{vq} V × $\mathcal{D}_{\text{final}}$ → Q，该过程根据用户需求和多视角图像V生成一组目标视觉问题Q = {*q1*； *q2*； … ； *qn*}。
- **视觉反馈生成***Fvf* V × Q → (*P* ； *F*)，该函数通过分析多视角图像V来回答问题Q，并输出明确的视觉反馈*F*及二元决策*P*。当*P*= 0时，将视觉反馈*F*返回至代码生成阶段以重新生成代码；当*P*= 1时，结果直接呈现给用户进行最终判断。用户对建模结果的满意度由*Sat* M → {0； 1}决定：若*Sat*(M) = 1，则将M添加至案例库K（即*Case′* =*Case* {M}）；若*Sat*(M) = 0，则重新进入代码生成阶段。

在整个过程中，除用户反馈链接（*R*和*Sat*）外，所有状态转换*Trans* *State* → *State*均由CADDesigner独立确定。

4. CAD代码生成范式

本节从四个方面详细阐述一种新型CAD代码生成范式：显式上下文指令范式（ECIP）、基于 ECIP 的CADAPI的实际实现方案、适配大语言模型的CAD代码设计方法，以及可复用建模知识库的构建。

4.1. 显式语境指令范式

近年来，许多研究都将**CadQuery**（CQ）代码作为CAD模型生成的中间表示形式。CadQuery通常采用Fluent API设计范式[8]，通过链式方法调用来表达操作步骤，从而实现高度可读且简洁的建模流程。该设计风格依赖于一个*隐式的内部上下文C*，用于追踪当前建模状态并在各操作之间自动传递。从形式上讲，每个操作f_k都会使用参数p_k更新隐藏状态*Ck*：

$$C_{k+1} = f_k(C_k, p_k). \tag{4}$$

CadQuery

```
result = (
    cq.Workplane("front")
    .box(2, 2, 0.5)
    .faces(">z")
    .workplane()
    .rect(1.5, 1.5, forConstruction=True)
    .vertices()
    .hole(0.125))
```

ECIP

```
hole_centers = [(x₁, y₁, z₁), ...]
result = make_box_rsolid(2, 2, 0.5)
holes = [
    make_cylinder_rsolid(..., ...)
    for hole center in hole_centers]
for hole in holes:
    result = cut_rsolid(result, hole)
```

Fig. 3. Code comparison between CadQuery (left) and ECIP (right). ECIP explicitly passes context and supports standard Python constructs, improving code clarity and flexibility.

A chained expression can therefore be written as:

$$C_n = f_n \circ f_{n-1} \circ \cdots \circ f_1(C_0). \tag{5}$$

While concise, this implicit state can make semantic dependencies less explicit, which may complicate automated code generation for LLMs that must track intermediate contexts. In contrast, low-level APIs like PythonOCC require explicit inputs for each operation, yielding clear semantics but verbose, fine-grained code that lacks high-level abstraction and usability.

To address these limitations, we introduce the **Explicit Context Imperative Paradigm (ECIP)**, which enforces explicit passing of all input objects and parameters to each operation. Formally, each operation g_k maps explicit input state S_k and parameters p_k to a new state S_{k+1}:

$$S_{k+1} = g_k(S_k, p_k). \tag{6}$$

Thus, the modeling workflow is represented as a sequence of explicit state transformations:

$$S_1 = g_1(S_0, p_1),\ S_2 = g_2(S_1, p_2),\ \ldots,\ S_n = g_n(S_{n-1}, p_n), \tag{7}$$

where all intermediate states S_k are explicit variables.

Fig. 3 illustrates a typical code comparison between CadQuery and ECIP. CadQuery (left) uses chained method calls that implicitly pass context, making the code compact but less flexible for inserting standard Python statements or loops. In contrast, ECIP (right) explicitly passes all inputs, allowing the use of standard Python control flow like `for` loops and variable assignments, which enhances modularity, readability, and ease of debugging. Thus, ECIP combines the clarity of low-level APIs with the expressiveness of high-level modeling, improving the accuracy and interpretability of agent-driven CAD code generation. This does not imply that CadQuery cannot support more explicit coding styles; rather, ECIP standardizes such explicitness in a form that is more suitable for LLM-oriented generation and refinement.

4.2. ECIP-compliant CAD API design

ECIP (referred to as SimpleCADAPI in the actual project) is designed as a command-style Python API built on top of CadQuery's `occ_impl.shapes` module. It serves as an LLM-oriented intermediate representation that remains semantically aligned with CadQuery. The geometric classes (e.g., `Vertex`, `Edge`, `Solid`) are implemented as lightweight wrappers over CadQuery objects, preserving full interoperability with native CadQuery operations.

The design follows the Open-Closed Principle: core geometric types are closed for modification, while operations and extensions are introduced through additional functions. Moreover, SimpleCADAPI is fully compatible with the NumPy library, enabling efficient mathematical computation and geometric transformation directly on CAD entities. It provides explicit input–output interfaces for each operation, facilitating LLM-driven code generation, iterative correction, and modular composition, while retaining the high-level modeling capabilities of CadQuery. SimpleCADAPI is organized into two main parts: the **Core** module and the **API** module.

4.2.1. Core module

The Core module encapsulates fundamental geometric types and primitive operations. By wrapping the underlying OpenCASCADE `TopoDS_Shape` classes, it establishes a clear hierarchy and semantic structure for geometric objects, while also managing coordinate systems, supporting tagging and metadata attachment, and enforcing strict type annotations and exception handling.

- **Geometric Object Hierarchy:** The Core module defines a layered geometric object model comprising the following primary classes, all inheriting from `TaggedMixin` to provide unified tagging and metadata management:

  (1) **Vertex**: 0-dimensional points in 3D space.
  (2) **Edge**: 1-dimensional curves connecting vertices.
  (3) **Wire**: Ordered collections of edges forming open or closed chains.
  (4) **Face**: 2-dimensional bounded surfaces defined by wires (outer boundaries and holes).
  (5) **Shell**: Collections of faces forming partial or complete shells.
  (6) **Solid**: Closed 3-dimensional volumes with definable volumetric properties.
  (7) **Compound**: Arbitrary aggregates of geometric entities for grouping or assembly construction.

- **Coordinate Systems and Working Planes:** ECIP employs a unified coordinate system framework to ensure consistency and precision in geometric modeling and transformation. This framework includes:

  (1) **CoordinateSystem**: Represents a 3D spatial frame with an explicit origin, X-axis direction, and normal vector, providing an abstraction for spatial transformations.
  (2) **WORLD_CS**: A global singleton coordinate system instance defining the default reference frame. It follows the right-hand rule with origin at $[0, 0, 0]$, the X-axis pointing along the default width direction, the Y-axis along the default height direction, and the Z-axis upward.
  (3) **SimpleWorkplane**: Manages local coordinate systems via context managers, enabling nested coordinate frames with defined origins and orientations. The returned workplane object follows the `_wp` naming convention when the variable is not directly referenced within the block, ensuring linter and LLM compatibility. This simplifies relative geometric operations and supports complex modeling workflows.

- **Metadata & Tagging:** ECIP unifies semantic tagging and geometric metadata through a lightweight tag–metadata layer. Each entity maintains user-visible tags and structured metadata, while runtime lineage is stored separately to avoid polluting modeling data. This design supports three complementary mechanisms. First, primitive constructors perform geometry-driven tagging. For example, `make_box_rsolid` automatically assigns face-level semantic tags such as "top", "bottom", "left/right", and "side" based on face normals, edge structure, and primitive type. Second, derived operations attach semantic tags via topology tracking after geometric or topological edits. ECIP wraps OpenCASCADE builders and queries `Modified()`, `Generated()`, and `IsDeleted()` to compute a `TopoDelta`. Resulting faces are then automatically assigned operation-aware semantic tags (e.g., `origin.tool`, `op.cut.generated`, `origin.body`). This approach avoids brittle index-based bookkeeping and preserves semantic lineage across boolean and feature operations. Third, ECIP stores structured geometric metadata alongside shapes, including primitive type, box size (x, y, z), cylinder axis and height, sphere center and radius, coordinate system snapshots, topological references (`topo_ref`), and compact selector hints (e.g., center, normal, area, length, volume, and bounding box). These metadata are serializable and directly consumable by QL predicates such as `ql.tag(...)` and `ql.meta(...)`.

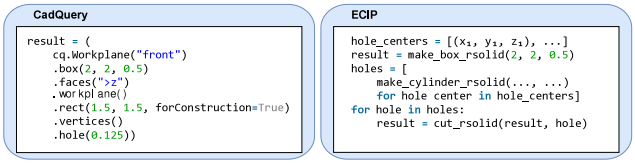

图3. CadQuery（左）与 ECIP（右）的代码对比。 ECIP 能明确传递上下文信息并支持标准Python语法结构，显著提升了代码的清晰度与灵活性。

因此，链式表达式可写为：

$$C_n = fn \circ fn_{-1} \circ \ldots \circ f_1(C_0): \tag{5}$$

尽管这种隐式状态简洁明了，但它会使语义依赖关系变得不那么显式，从而可能增加需要追踪中间上下文的大型语言模型（LLM）自动化代码生成的复杂性。相比之下，像PythonOCC这样的低级API要求每个操作都需提供显式输入，虽然能确保语义清晰，但生成的代码冗长且细节繁多，缺乏高层次抽象性和易用性。

为解决这些局限性，我们提出了**显式上下文指令范式（ECIP）**，该范式强制要求将所有输入对象和参数明确传递给每个操作。形式上，每个操作g_k都将显式的输入状态S_k及参数p_k映射到新的状态S_{k+1}：

$$S_{k+1} = g_k(S_k, p_k). \tag{6}$$

因此，建模工作流程可表示为一系列明确的状态转换序列：

$$S_1 = g\ 1(S_0; p\ 1)\ ; S_2 = g\ 2(S_1; p\ 2)\ ; \ldots, Sn = gn\ (Sn_{-1}; pn)\ ; \tag{7}$$

其中所有中间状态Sk均为显式变量。

图3展示了CadQuery与 ECIP 之间的典型代码对比。CadQuery（左）采用链式方法调用方式，隐式传递上下文信息，虽然代码简洁但难以灵活插入标准Python语句或循环结构；相比之下， ECIP（右）明确传递所有输入参数，支持使用for循环、变量赋值等标准Python控制流结构，从而提升模块化程度、可读性及调试便利性。因此， ECIP 既保留了低级API的清晰性，又兼具高级建模语言的表达能力，显著提升了基于智能体的CAD代码生成的准确性和可解释性。这并不意味着CadQuery无法支持更显式的编程风格——相反， ECIP 将这种显式编码形式标准化为更适合大语言模型驱动的代码生成与优化方案。

4.2. 符合 ECIP 标准的CAD API设计

ECIP（在实际项目中称为SimpleCADAPI）是一款基于CadQuery的 occ_impl： 形状模块构建的命令式Python应用程序接口。它作为面向大型语言模型的中间表示层，同时保持与CadQuery在语义上的完全一致性。几何类（如顶点、边、实体）均以轻量级封装形式实现于CadQuery对象之上，确保与原生CadQuery操作具备完全互操作性。

该设计遵循开闭原则：核心几何类型不可修改，而操作与扩展功能则通过附加函数实现。此外，SimpleCADAPI与NumPy库完全兼容，可直接在CAD实体上高效执行数学计算和几何变换。它为每个操作提供明确的输入输出接口，支持基于大语言模型的代码生成、迭代修正及模块化组合，同时保留了CadQuery的高级建模能力。SimpleCADAPI由两个主要部分组成：**核心**模块和**API**模块。

4.2.1. 核心模块

核心模块封装了基本几何类型和基础运算操作。通过封装底层 OpenCASCADE TopoDS_Shape 类，它为几何对象建立了清晰的层次结构与语义框架，同时管理坐标系、支持标签标注与元数据附加，并强制执行严格的类型注释及异常处理机制。

- **几何对象层次结构：**核心模块定义了一个分层化的几何对象模型，包含以下主要类别，所有类均继承自TaggedMixin以实现统一的标签和元数据管理：

  (1) 顶点：三维空间中的零维点。

  (2) 边：连接顶点的一维曲线。

  (3) 线：由边按特定顺序排列形成的开放或闭合链结构。

  (4) 面：由线段（外部边界和孔洞）定义的二维有界曲面。

  (5) 外壳：由多个面组成的、构成部分或完整外壳的集合。

  (6) 实心：具有可定义体积特性的封闭三维空间。

  (7) 复合体：用于分组或装配构建的几何实体任意组合。

- **坐标系与工作平面：**ECIP 采用统一的坐标系框架，以确保几何建模与变换过程中的统一性与精确性。该框架包括：

  (1) 坐标系：表示一个具有明确原点、X轴方向及法向量的三维空间坐标系，为空间变换提供抽象化描述。

  (2) WORLD_CS：定义默认参考系的全局单例坐标系实例。该坐标系遵循右手定则，原点位于$[0, 0, 0]$，X轴沿默认宽度方向，Y轴沿默认高度方向，Z轴向上。

  (3) SimpleWorkplane：通过上下文管理器管理局部坐标系，支持具有定义原点和方向的嵌套坐标系。当该变量未在块内直接引用时，返回的工作平面对象遵循 _wp 命名规范，确保与Linter及LLM兼容。这简化了相对几何运算，并支持复杂的建模工作流程。

- **元数据与标记：**ECIP 通过轻量级的标签-元数据层统一了语义标记与几何元数据。每个实体均保留用户可见的标签及结构化元数据，而运行时的变更轨迹则被独立存储，以避免污染建模数据。该设计支持三种互补机制：首先，基本构造函数可基于几何特征进行标记处理——例如 make_box_rsolid 会根据面法线、边结构及图元类型自动为面分配“顶部”、“底部”、“左右”或“侧面”等语义标签；其次，在执行几何或拓扑修改后，派生操作可通过拓扑追踪附加语义标签； ECIP 封装了OpenCASCADE的Modified()、Generated()和IsDeleted()函数来计算TopoDelta值，并自动为生成的新面分配包含操作信息的语义标签（如origin:tool、op:cut:generated、origin:body）。这种设计避免了依赖索引的脆弱记录机制，并确保布尔运算与特征操作过程中语义传承的一致性。第三， ECIP 在存储形状的同时还存储结构化的几何元数据，包括基本类型、矩形尺寸（x; y; z）、圆柱轴线与高度、球体中心及半径、坐标系快照、拓扑参照（topo_ref）以及紧凑选择器提示信息（如中心点、法线、面积、长度、体积和边界框）。这些元数据可序列化，并能直接被QL谓词（如ql:tag（:::）和ql:meta（:::））处理。

Table 1
Summary of core CAD modeling functions by category.

Category	Examples	Count
Creation	`make_box_rsolid`, `make_cylinder_rsolid`, `make_circle_rwire`	25
Transformation	`translate_shape`, `rotate_shape`, `mirror_shape`	3
3D Operations	`extrude_rsolid`, `revolve_rsolid`, `loft_rsolid`, `sweep_rsolid`	4
Boolean	`union_rsolidlist`, `cut_rsolidlist`, `intersect_rsolidlist`	3
Advanced Features	`fillet_rsolid`, `chamfer_rsolid`, `shell_rsolid`, `helical_sweep_rsolid`	4
Pattern	`linear_pattern_rsolidlist`, `radial_pattern_rsolidlist`	2
Tagging & Selection	`set_tag`, `ql.tag`, `ql.meta`, `ql.where`, `ql.order_by`	14
Export	`export_step`, `export_stl`	2

- **Query Language (QL) for Geometric Selection:** SimpleCADAPI provides a dedicated query language module (`ql`) for declarative geometric selection. QL selectors are fully serializable, enabling transparent logging and replay of selection operations. A selector is constructed by chaining entry-point functions (e.g., `ql.edges()`, `ql.faces()`), predicates (e.g., `ql.tag()`, `ql.meta()`), ordering keys (e.g., `ql.center_axis()`, `ql.geo()`), and cardinality constraints (e.g., `.take(n)`, `.exactly(n)`). Crucially, QL unifies two complementary selection modes: querying semantic lineage derived from `TopoDelta`-based tracking, and direct geometric selection based on shape properties. The example below illustrates both behaviors within a unified workflow.
- **Example Usage:**

```python
from simplecadapi import *

# Base solid with user tags and metadata
base = make_box_rsolid(width=12, height=8, depth=4)
base.add_tag("base_frame")
base.set_metadata("material", "aluminum")

# Add a boss in a local workplane
with SimpleWorkplane(origin=(0, 0, 4)) as _wp:
    boss = make_cylinder_rsolid(radius=2.5, height=2)
model = union_rsolidlist(base, boss)

# Through-hole cut (TopoDelta tags attached)
tool = make_cylinder_rsolid(radius=1.0, height=6)
model = cut_rsolidlist(model, tool)

# Semantic selection via topology tracking
new_cut_faces = (
    ql.faces().where(
        ql.and_(
            ql.tag("origin.tool"),
            ql.meta("track.event", "==", "generated"))
    ).resolve(model)
)
# Geometry-based selection (order-independent)
top_hole_edge = (
    ql.edges().where(ql.curve_type("circle"))
    .order_by(ql.center_axis("z"), desc=True)
    .take(1).exactly(1)
)
# Apply chamfer
result = chamfer_rsolid(model, top_hole_edge,
        distance=0.5)
```

4.2.2. API module

The API module builds upon the Core module and provides higher-level composite operations and LLM-friendly interfaces for advanced modeling tasks. Its design mainly includes the following two aspects:

- **Core Modeling Functions**: A comprehensive set of over 50 CAD modeling functions organized into categories such as creation, transformation, 3D operations, boolean operations, advanced features, pattern generation, tagging & selection, and export. These functions allow users or agents to flexibly compose complex modeling procedures while maintaining explicit state transitions and clear semantics. For user-facing convenience, some primitive constructors expose semantic dimension names such as `width`, `height`, and `depth`, but the API documentation binds them explicitly to the world axes as x, y, and z, respectively. A summary of the function categories, representative examples, and counts is shown in Table 1.
- **Automation and Self-Maintenance**: To maintain consistency and support continuous evolution, the API module incorporates an automation pipeline with the following functions:

  (1) Source Code Parsing: Automatically parses user-defined Python source files to extract and integrate new or updated modeling functions into the public API.
  (2) Function Categorization: Scans and categorizes source code to generate up-to-date API initialization files and alias mappings for consistent public exposure.
  (3) Documentation Generation: Extracts function signatures and docstrings to produce structured markdown documentation, improving usability and maintainability.
  (4) Knowledge Base Synchronization: Synchronizes generated documentation with an external knowledge base, enabling keyword-based indexing and improved traceability.

4.3. LLM-friendly CAD code design

Beyond the ECIP paradigm and its API implementation, we incorporate several complementary design features aimed at improving LLM agents' ability to accurately generate and debug CAD modeling code.

4.3.1. Structured error information

The error handling mechanism of CadQuery typically produces generic or partially unstructured error messages, which can make automated debugging and code correction more challenging in some scenarios. To facilitate more effective guidance for LLM-driven code generation, ECIP introduces fine-grained structured error messages at the atomic operation level, represented as triples:

$$\texttt{ErrMsg} = (\texttt{ErrCau}, \texttt{ErrLoc}, \texttt{CorrAct}), \tag{8}$$

where `ErrCau` identifies the root cause of the failure, `ErrLoc` specifies the exact location of the error in the script, and `CorrAct` suggests potential corrective actions. This structured format provides actionable feedback that LLMs can leverage to improve fault localization and enable automated code repair.

4.3.2. Explicit type annotations

Type ambiguity commonly leads to erroneous code generation, particularly when dealing with closely related geometric entities such as Wire and Edge. ECIP mitigates this issue by enforcing explicit type annotations through a disciplined naming convention for functions, following the pattern:

$$\texttt{ActionName\_rReturnType}, \tag{9}$$

表 1
按类别划分的核心CAD建模功能概览。

类别	示例	计数
创世	make_box_rsolid, make_cylinder_rsolid, make_circle_rwire	25
变换	转换形状、旋转形状、镜像形状	3
3D操作	extrude_rsolid, revolve_rsolid, loft_rsolid, sweep_rsolid	4
布尔数学体系的	union_rsolidlist, cut_rsolidlist, intersect_rsolidlist	3
高级功能	fillet_rsolid, chamfer_rsolid, shell_rsolid, helical_sweep_rsolid	4
模式	linear_pattern_rsolidlist, radial_pattern_rsolidlist	2
标记与选择	设置标签，ql：标签，ql：元数据，ql：条件，ql：排序方式	14
导出	导出步骤， export_stl	2

- **几何选择查询语言（QL）：** SimpleCADAPI为声明式几何选择提供了专用的查询语言模块（QL）。QL选择器完全可序列化，能够实现选择操作的透明日志记录与回放。选择器由入口函数（如ql:edges()、ql:faces()）、谓词（如ql:tag()、ql:meta()）、排序键（如ql:center_axis()、ql:geo()）以及基数约束条件（如:take(n)、exactly(n)）串联构成。关键在于，QL整合了两种互补的选择方式：基于TopoDelta追踪技术推导出的语义溯源查询，以及基于形状属性的直接几何选择。下例展示了这两种方式在统一工作流程中的应用。
- **示例用法：**

```
from simplecadapi import *

# 包含用户标签和元数据的基础实体
base = make_box_rsolid(width=12, height=8, depth=4)
base.add_tag("base_frame")
base.set_metadata("material", "aluminum")

# Add a boss in a local workplane
with SimpleWorkplane(origin=(0, 0, 4)) as _wp:
   boss = make_cylinder_rsolid(radius=2.5, height=2)
model = union_rsolidlist(base, boss)

# Through-hole cut (TopoDelta tags attached)
tool = make_cylinder_rsolid(radius=1.0, height=6)
model = cut_rsolidlist(model, tool)

# Semantic selection via topology tracking
new_cut_faces = (
   ql.faces().where(
      ql.and_(
         ql.tag("origin.tool"),
         ql.meta("track.event", "==", "generated"))
   ).resolve(model)
)
# Geometry-based selection (order-independent)
top_hole_edge = (
   ql.edges().where(ql.curve_type("circle"))
   .order_by(ql.center_axis("z"), desc=True)
   .take(1).exactly(1)
)
# Apply chamfer

结果 = chamfer_rsolid (模型， 顶部孔边缘， 距离=0.5)
```

4.2.2. API模块

API模块基于Core模块构建，为高级建模任务提供更高层次的复合操作功能及符合大语言模型（LLM）特性的接口。其设计主要涵盖以下两个方面：

- **核心建模功能**：包含超过50项CAD建模功能的完整集合，这些功能按创建、变换、三维操作、布尔运算、高级特性、图案生成、标记与选择以及导出等类别划分。这些功能使用户或智能体能够灵活构建复杂的建模流程，同时保持明确的状态转换和清晰的语义关系。为提升用户体验便利性，部分基础构造函数会提供宽度、高度、深度等语义维度名称，但API文档中将其分别明确定义为x、y和z三个世界坐标轴。各功能类别、典型示例及数量统计详见表1。
- **自动化与自主维护**：为确保一致性并支持持续演进，该API模块集成了具备以下功能的自动化流程：

  (1) 源代码解析：自动解析用户定义的Python源文件，提取并整合新的或更新的建模函数至公共API中。

  (2) 功能分类：扫描并分类源代码，生成最新的API初始化文件及别名映射关系，以确保公共接口的一致性。

  (3) 文档生成：提取函数签名和文档字符串，生成结构化的Markdown文档，提升可用性与可维护性。

  (4) 知识库同步：将生成的文档与外部知识库同步，支持基于关键词的索引功能并提升可追溯性。

4.3. 符合 LLM 标准的 CAD 代码设计

除了 ECIP 范式及其API实现之外，我们还整合了多项互补性设计功能，旨在提升LLM智能体准确生成和调试CAD建模代码的能力。

4.3.1. 结构化错误信息

CadQuery的错误处理机制通常生成通用或部分非结构化的错误信息，这在某些场景下会增加自动化调试和代码修正的难度。为更有效地指导基于大语言模型（LLM）的代码生成， ECIP 在原子操作层面引入了细粒度的结构化错误信息，这些信息以三元组形式表示：

$$\mathrm{ErrMsg} = (\mathrm{ErrCau}, \mathrm{ErrLoc}, \mathrm{CorrAct}), \tag{8}$$

其中ErrCau标识故障的根本原因，ErrLoc指定脚本中错误的确切位置，CorrAct则提出潜在的修正措施。这种结构化格式提供了可操作的反馈信息，大型语言模型（LLM）可利用这些信息来提升故障定位能力并实现代码自动修复。

4.3.2. 显式类型注释

类型歧义常导致代码生成错误，尤其是在处理线（Wire）和边（Edge）等密切相关的几何实体时。 ECIP 通过采用规范化的函数命名约定来强制要求显式类型注解，从而有效解决这一问题：

$$\mathrm{ActionName\_rReturnType}, \tag{9}$$

where `ActionName` denotes the CAD operation, `r` is a fixed prefix indicating "returns", and `ReturnType` explicitly declares the type of the returned object. This ensures that even without inspecting documentation, the function signature itself conveys type information, reducing ambiguity during LLM inference. Each function is also accompanied by comprehensive, type-annotated documentation. This explicit type information reduces ambiguity and improves code generation accuracy by LLMs.

4.3.3. Self-evolving composite operations

To facilitate knowledge accumulation and abstraction, ECIP supports the definition of *composite operations*—higher-level constructs formed by encapsulating sequences of atomic operations. These composites are saved as new operations within the language, enabling agents to invoke them directly in subsequent tasks.

This self-evolving mechanism effectively expands the language's set of available operations, allowing reuse of complex modeling patterns such as screws, flanges, or other parametric features. In principle, this can improve one-shot generation success by allowing LLM agents to reuse accumulated domain knowledge more efficiently.

4.4. Knowledge base for code framework

LLMs possess strong generative and reasoning abilities but often struggle with the fine-grained semantic requirements of CAD APIs, such as precise parameter formats, geometric constraints, and operation-specific return types. These limitations frequently lead to misinterpreted command signatures or hallucinated inputs, resulting in unstable code generation.

To provide explicit, domain-grounded guidance during modeling, we construct a structured knowledge base $\mathcal{K}$ coupled with retrieval-augmented generation (RAG). Retrieved function annotations and example cases supply accurate API semantics that complement the LLM's implicit priors, helping improve generation reliability. The knowledge base $\mathcal{K}$ consists of two key components:

- **Function Annotations** (`Anno`): Semantic descriptions of API functions, including parameter formats, return types, and usage notes;
- **Case Examples** (`Case`): Initially composed of manually selected examples and gradually expanded during use.

To automate the construction of $\mathcal{K}$, we design a rule-based pipeline that extracts and organizes information directly from the source code. Specifically, we collect function signatures, parameter types, and return types from Python annotations and type hints, and align them with the corresponding natural language descriptions provided in function-level docstrings. This process yields a structured and semantically consistent API documentation corpus.

4.4.1. Structured representation for retrieval

We further organize the extracted knowledge into a standardized documentation format to support RAG. Each API entry is represented as a structured Markdown document following a unified template, including function signatures with type annotations, purpose descriptions, parameter specifications, return values, possible exceptions, and practical usage examples. All entries are presented using second-level Markdown headings for structural consistency.

This standardized representation improves retrieval accuracy, reduces ambiguity in natural language queries, and enhances interpretability for both models and developers. For example, a CAD operation such as `chamfer_rsolid` specifies its input types (e.g., a `Solid` object, a list of `Edge` objects, and a `float` distance parameter) together with usage examples, enabling the retriever to match queries with correct API semantics and usage patterns.

4.4.2. Semantic-friendly chunking

When building the vectorized representation of $\mathcal{K}$, we employ a customized **semantic-friendly chunking** strategy based on the standardized Markdown structure of API documents. Specifically, each second-level heading (##) is treated as a boundary defining a semantic block. This design ensures **semantic completeness** (each chunk forms a self-contained unit such as a method or parameter description), **semantic independence** (different chunks can be processed and retrieved modularly), and **structural alignment** (the chunking respects the inherent document structure and avoids arbitrary or mid-sentence splits).

Compared to naive fixed-length chunking, this strategy preserves the logical integrity of API documentation and ensures that retrieved content remains coherent and directly usable during generation.

To further improve retrieval precision, we append **anchor keywords** to each chunk, which explicitly summarize its semantic context. These anchors consist of the API name, the section label, and a small set of representative content tokens. For example, a chunk corresponding to the purpose section of `chamfer_rsolid` may include:

`[chamfer_rsolid, purpose, chamfer, edge, transition]`

These anchors act as explicit semantic signals that enhance embedding-based retrieval and relevance scoring, improving both recall and precision while preserving the original document semantics. Overall, this design ensures that retrieval at inference time yields coherent and contextually complete knowledge snippets, effectively guiding downstream code generation and improving the robustness of LLM-based CAD reasoning.

5. Experiments and comparison

This section details the implementation of CADDesigner and the comprehensive experimental evaluation conducted to assess its performance in CAD modeling code generation.

5.1. Implementation details

We evaluate our CAD modeling code generation algorithm on a workstation with an 8-core Intel CPU under Python 3.12. The system is implemented on a customized agent framework that leverages retrieval-augmented generation (RAG) via RAGFlow. For embedding-based retrieval, we utilize the embedding model bge-large-zh-v1.5 to encode knowledge base documents. The retrieval employs a hybrid similarity scoring method, where vector similarity and keyword-based similarity are combined with weights of 0.6 and 0.4, respectively. The top 3 most relevant documents (top-k=3) are retrieved for downstream generation. For LLM services, the primary agent and the code generation tool (T_2) employ Claude-4-Sonnet, while requirement refinement (T_1) and both visual feedback sub-tools (visual question generation and visual feedback generation) are based on Gemini-2.5-Flash. Inference uses a temperature of 0.7 and nucleus sampling (top-p) of 0.9 to balance diversity and coherence. All experiments are conducted without the user feedback loops (R and Sat), so the reported results do not include online case accumulation driven by user approval.

5.2. Dataset

Our experiments are conducted on the large-scale DeepCAD dataset [1], which contains approximately 178K parametric CAD models represented in a sketch-and-extrude format. Following SkexGen [10], we first remove duplicate models to reduce redundancy. From the training set, we randomly select $1K$ models to initialize the RAG knowledge base. In addition, we develop a conversion script to transform the parametric command sequences of each model into ECIP code snippets, which are included in the RAG knowledge base as `Case`. For evaluation, we construct two test subsets from the de-duplicated DeepCAD test set.

其中 `ActionName` 表示 CAD 操作，`r` 是表示“返回”的固定前缀，而 `ReturnType` 则明确声明返回对象的类型。这确保即使不查阅文档，函数签名本身也能传达类型信息，从而降低语言模型推理过程中的歧义性。每个函数均配有详尽且带有类型注释的文档说明。这种显式的类型信息不仅能减少歧义，还能提升语言模型生成代码的准确性。

4.3.3. 自演化复合运算

为促进知识积累与抽象化， ECIP 支持定义*复合操作*——即通过封装一系列原子操作而形成的高级结构。这些复合操作会在语言中作为新操作保存，使智能体能够在后续任务中直接调用它们。

这种自我演进机制有效扩展了语言可支持的操作范围，使得螺钉、法兰或其他参数化特征等复杂建模模式能够被重复使用。从原理上讲，这可通过让大语言模型代理更高效地复用积累的领域知识，从而提升单次生成的成功率。

4.4. 代码框架知识库

LLM具备强大的生成能力和推理能力，但往往难以满足CAD API对细粒度语义要求的规范，例如精确的参数格式、几何约束条件以及特定操作对应的返回类型。这些局限性常导致命令签名被误读或输入数据被错误解析，从而引发代码生成不稳定的问题。

为在建模过程中提供明确且基于领域知识的指导，我们构建了一个结构化知识库 结合检索增强生成（RAG）技术。检索到的功能注释和示例案例提供了精确的API语义信息，与大语言模型的隐含先验知识形成互补，从而提升生成结果的可靠性。该知识库 包含两个核心组成部分：

- **函数注释**（Anno）：API函数的语义描述，包括参数格式、返回类型及使用说明；
- **案例示例**（案例集）：最初由人工精选的示例组成，并在使用过程中逐步扩展。

为实现 ，构建过程的自动化，我们设计了一套基于规则的流程，可直接从源代码中提取并整理信息。具体而言，我们从Python注释和类型提示中收集函数签名、参数类型及返回类型，并将其与函数级文档字符串中的对应自然语言描述进行匹配。该流程最终生成结构化且语义一致的API文档语料库。

4.4.1. 用于检索的结构化表示

我们进一步将提取的知识整理成标准化的文档格式以支持RAG功能。每个API条目均采用统一模板构建为结构化Markdown文档，内容包括带类型注释的功能签名、用途说明、参数规范、返回值、可能异常情况以及实际使用示例。所有条目均使用二级Markdown标题进行呈现，以确保结构一致性。

这种标准化表示方式提高了检索准确性，减少了自然语言查询中的歧义性，并增强了模型与开发者的可解释性。例如，诸如 chamfer_rsolid 之类的CAD操作会明确指定其输入类型（如实体对象、边缘对象列表以及浮点距离参数）并附带使用示例，从而使检索系统能够根据正确的API语义和使用模式匹配查询。

4.4.2. 语义友好的分块处理

在构建 ，的向量化表示时，我们采用了一种基于API文档标准化Markdown结构的定制化**语义友好型分块策略**。具体而言，每个二级标题（##）均被视为界定语义区块的边界。这种设计确保了**语义完整性**（每个分块构成自成一体的单元，如方法或参数描述）、**语义独立性**（不同分块可模块化处理与提取）以及**结构对齐性**（分块方式尊重文档固有结构，避免随意或跨句分割）。

与传统的固定长度分块方法相比，该策略能够保持API文档的逻辑完整性，并确保提取的内容在生成过程中保持连贯且可直接使用。

为进一步提升检索精度，我们在每个数据片段中添加了**锚定关键词**，这些关键词能明确概括其语义上下文。这些锚点包含API名称、章节标题以及一组代表性内容标记。例如，对应 chamfer_rsolid “目的”章节的数据片段可能包含以下内容：

[chamfer_rsolid 、用途、倒角、边缘、过渡]

这些锚点作为明确的语义信号，能够增强基于嵌入的检索能力和相关性评分，既提升召回率与精确度，又保留了原始文档的语义特征。总体而言，该设计确保推理阶段的检索结果能生成连贯且符合上下文的知识片段，有效指导后续代码生成过程，并增强基于大语言模型（LLM）的计算机辅助设计（CAD）推理系统的鲁棒性。

5. 实验与比较

本节详细阐述了CADDesigner的实现方案以及为评估其在CAD建模代码生成方面的性能而开展的全面实验评估。

5.1. 实施细节

我们在配备8核英特尔CPU的工作站上，使用Python 3.12环境对CAD建模代码生成算法进行了评估。该系统基于定制化的智能体框架实现，并利用RAGFlow提供的检索增强生成（RAG）技术。在基于嵌入的检索方面，我们采用bge-large-zh-v1.5嵌入模型对知识库文档进行编码；检索过程采用混合相似度评分方法，将向量相似度与关键词相似度分别以0.6和0.4的权重相结合，最终提取出最相关的前3篇文档（top-k=3）用于后续生成任务。对于大语言模型服务，主智能体及代码生成工具（T_2）采用Claude-4-Sonnet模型，需求细化模块（T_1）以及两个视觉反馈子工具（可视化问题生成与可视化反馈生成）均基于Gemini-2.5-Flash模型。推理过程中设置温度参数为0.7、核采样率（top-p）为0.9，以平衡多样性和连贯性。所有实验均未包含用户反馈循环（R和sat），因此报告结果不包含由用户批准驱动的在线案例积累影响。

5.2. 数据集

我们的实验基于大规模DeepCAD数据集[1]进行，该数据集包含约17.8万个采用草图-拉伸格式表示的参数化CAD模型。遵循SkexGen[10]的方法，我们首先去除重复模型以降低冗余度；从训练集中随机选取1000个模型作为RAG知识库的初始化样本；同时开发了转换脚本，将每个模型的参数化命令序列转化为 ECIP 代码片段，并将其作为Case单元纳入RAG知识库。为评估性能，我们从去重后的DeepCAD测试集中构建了两个测

The first subset contains 200 models, selected using uniform stratified sampling based on the number of commands in the sequence (1–10, 11–20, 21–30, 31–40, 41+) to ensure coverage across different complexity levels. The second subset contains $1K$ models randomly sampled from the same de-duplicated test set and is used for comparison with prior text-to-CAD methods. For each sampled model, a single representative image is rendered from a fixed camera viewpoint to provide visual input for the experiments, while the textual inputs are derived from the abstract-level descriptions provided in the DeepCAD dataset.

5.3. Evaluation metrics

To assess the geometric fidelity of the generated CAD models, we adopt several standard shape similarity metrics. Let $\mathcal{S}$ denote the reference models and $\mathcal{G}$ represent the generated models, with $\mathcal{X}$ and $\mathcal{Y}$ representing their corresponding point clouds. These metrics evaluate the agreement between generated and ground-truth geometries from volumetric and point-wise perspectives.

Intersection over Union (IoU) is utilized to measure the similarity between generated models and the ground truth.

$$\mathrm{IoU}(\mathcal{G}, \mathcal{S}) = \frac{\mathcal{G} \cap \mathcal{S}}{\mathcal{G} \cup \mathcal{S}}, \tag{10}$$

where $\mathcal{G} \cap \mathcal{S}$ denotes the overlapping volume between the reference and generated models, and $\mathcal{G} \cup \mathcal{S}$ represents their combined volumetric union. A value of 1 indicates perfect alignment, while 0 signifies no overlap.

Chamfer Distance (CD) calculates point-wise proximity between point clouds sampled from $\mathcal{X}$ and $\mathcal{Y}$:

$$\begin{aligned}\mathrm{CD}(\mathcal{X}, \mathcal{Y}) = &\frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} \min_{y \in \mathcal{Y}} \|x - y\|_2^2 \\ &+ \frac{1}{|\mathcal{Y}|} \sum_{y \in \mathcal{Y}} \min_{x \in \mathcal{X}} \|y - x\|_2^2.\end{aligned} \tag{11}$$

Hausdorff Distance (HD) is a metric for quantifying the similarity between two point sets, primarily used for evaluating the resemblance of object contours.

$$\begin{aligned}\mathrm{HD}(\mathcal{X}, \mathcal{Y}) = \max \Big\{ &\sup_{x \in \mathcal{X}} \inf_{y \in \mathcal{Y}} d(x, y), \\ &\sup_{y \in \mathcal{Y}} \inf_{x \in \mathcal{X}} d(x, y) \Big\},\end{aligned} \tag{12}$$

where $d(x, y)$ is the Euclidean distance between points x and y, sup (supremum) and inf (infimum) represent the least upper bound and greatest lower bound, respectively.

To ensure fair and reproducible comparison, all geometric metrics are evaluated under standardized **metric computation protocols**, including normalization, alignment, voxelization, and point cloud sampling, which are applied consistently across all methods and baselines:

- **Normalization and Alignment:** All meshes and sampled point clouds are first normalized to fit within a unit cube $[-0.5, 0.5]^3$. For pairwise comparisons, rigid alignment between predicted and ground-truth shapes is performed using the Iterative Closest Point (ICP) algorithm to eliminate differences due to translation and rotation.
- **IoU Voxelization:** For IoU computation, both meshes are voxelized using a fixed voxel size of 0.02. The voxel grid is defined over the union of the bounding boxes of both models, with an additional margin of two voxels to ensure full coverage.
- **Point Cloud Sampling:** For Chamfer Distance (CD) and Hausdorff Distance (HD), 2048 points are uniformly sampled from the surface of each mesh. This sampling density is kept consistent across all methods.

In addition, we introduce several process-oriented metrics to analyze modeling stability, reliability, and efficiency under ablation settings: **Pass@1** denotes the percentage of cases where valid code is generated on the first attempt, reflecting generation accuracy. **AVG Re** (Average Retry) captures the mean number of retries required during the iterative correction loop, indicating convergence efficiency. **SUC** denotes the proportion of cases where a syntactically valid CAD model is successfully generated, i.e., models that can be executed and rendered without errors. Note that a syntactically valid model is not necessarily semantically aligned with the input description. **Tokens** measures the average number of tokens consumed to generate a single CAD model. **Latency** measures the average time required to generate a single CAD model.

Table 2
Ablation study of ECIP design components on 200 test models.

	Pass@1↑	AVG Re↓	SUC↑
ECIP	**0.45**	**1.86**	**100.0%**
ECIP w/o Err	0.41	2.62	81.5%
ECIP w/o Type	0.32	2.30	90.5%

5.4. Comprehensive ablation study

To comprehensively verify the design choices and the effectiveness of different components in our CADDesigner agent, we conduct four groups of ablation studies using the 200-model test subset described in Section 5.2: one focusing on the ECIP design, another evaluating the impact of CAD tools and input modalities, a third analyzing the effect of different LLM backends, and a fourth investigating how model complexity influences token usage and generation latency. Unless otherwise specified, all experiments are conducted with text-only input.

5.4.1. Effect of ECIP design components

To illustrate the benefits of the ECIP design choices, such as the error handling mechanism that is friendly to LLMs and the overall architecture of the ECIP system itself, we evaluate three ECIP variants (full ECIP, ECIP without the detailed error system, and ECIP without the return type annotations).

The results in Table 2 show that the full ECIP configuration achieves the best overall performance among all variants. When the error handling mechanism is removed (ECIP w/o Err), the Pass@1 decreases slightly from 0.45 to 0.41, while the AVG Re increases from 1.86 to 2.62, and SUC drops to 81.5%. This suggests that although the agent can still generate correct code initially, the lack of detailed feedback slows down convergence during iterative correction and reduces the overall success rate. In contrast, removing type annotations (ECIP w/o Type) causes Pass@1 to drop significantly from 0.45 to 0.32, indicating that the initial code generation is more prone to semantic errors without explicit type guidance. The AVG Re (2.30) is higher than full ECIP, but still lower than ECIP without error handling, and SUC also drops to 90.5%. This shows that type annotations help encode critical modeling intent into the prompt, yet some type errors can still be corrected through retries. Overall, we observe a clear division of roles: error handling helps the agent quickly recover from mistakes during iterations, and type annotations improve first-attempt accuracy by reducing semantic ambiguity.

5.4.2. Effect of CAD tools and input modalities

To further analyze the individual contributions of the key components in CADDesigner, we evaluate seven variants, labeled A to G, which respectively correspond to: (A) removal of function annotations `Anno` from the knowledge base, (B) removal of basic model cases `Case`, (C) disabling the requirement refinement tool T_1, (D) disabling the visual feedback tool T_4, (E) the full CADDesigner pipeline with text-only input, (F) full pipeline with image-only input, and (G) full pipeline with combined text and image input. Except for variants F and G, all others use text input exclusively.

Table 3 summarizes the quantitative results across multiple evaluation metrics. In Model A, removing semantic API annotations (`Anno`)

试子集：第一个子集包含200个模型，根据命令序列长度（1–10、11–20、21–30、31–40、41+）采用均匀分层抽样法选取，以确保覆盖不同复杂度水平；第二个子集则从同一去重测试集中随机抽取1000个模型，用于与现有文本转CAD方法进行对比。对于每个采样的模型，均从固定相机视角生成一张代表性图像作为实验的视觉输入；文本输入则源自DeepCAD数据集中提供的抽象层面描述。

5.3. 评估指标

为评估生成CAD模型的几何保真度，我们采用了多种标准形状相似性指标。设S表示参考模型，G表示生成模型， 和y分别代表它们对应的点云。这些指标从体积测量和逐点分析两个角度评估生成模型与真实几何数据之间的吻合程度。

交并比（IoU）用于衡量生成模型与真实标注结果之间的相似度。

$$\mathrm{IoU}(\mathcal{G}, \mathcal{S}) = \frac{\mathcal{G} \cap \mathcal{S}}{\mathcal{G} \cup \mathcal{S}}, \tag{10}$$

其中 G ∩ S 表示参考模型与生成模型之间的重叠体积，G S 则表示两者的总体积并集。值为 1 表示完全对齐，而 0 表示无重叠。

倒角距离（CD）用于计算从 轴和y轴采样点云之间的逐点接近度：

$$\begin{aligned}\mathrm{CD}(\mathcal{X}, \mathcal{Y}) = &\frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} \min_{y \in \mathcal{Y}} \|x - y\|_2^2 \\ &+ \frac{1}{|\mathcal{Y}|} \sum_{y \in \mathcal{Y}} \min_{x \in \mathcal{X}} \|y - x\|_2^2.\end{aligned} \tag{11}$$

Hausdorff距离（HD）是一种用于量化两个点集相似性的度量指标，主要应用于评估物体轮廓的相似性。

$$\begin{aligned}\mathrm{HD}(\mathcal{X}, \mathcal{Y}) = \max \Big\{ &\sup_{x \in \mathcal{X}} \inf_{y \in \mathcal{Y}} d(x, y), \\ &\sup_{y \in \mathcal{Y}} \inf_{x \in \mathcal{X}} d(x, y) \Big\},\end{aligned} \tag{12}$$

其中*d*(*x*，*y*)表示点*x*与*y*之间的欧几里得距离，sup（上确界）和inf（下确界）分别表示最小上界和最大下界。

为确保比较的公平性和可重复性，所有几何指标均采用标准化**度量计算协议**进行评估，包括归一化、对齐、体素化及点云采样等步骤，并在所有方法及基线方案中保持一致应用。

- **标准化与对齐：**所有网格和采样点云首先均需进行标准化处理，使其适配于单位立方体$[-0.5, 0.5]^3$内。为进行成对比较，采用迭代最近点（ICP）算法对预测形状与真实形状进行刚性对齐，以消除由平移和旋转引起的差异。

- **IoU体素化：**在计算IoU时，两个模型的网格均采用固定体素尺寸0.02进行体素化处理。体素网格定义于两个模型边界框的并集上，并额外增加两个体素作为余量以确保完全覆盖。

- **点云采样：**对于倒角距离（CD）和豪斯多夫距离（HD），从每个网格表面均匀采样2048个点。该采样密度在所有方法中保持一致。

此外，我们引入了若干面向流程的指标来分析在消融设置下的建模稳定性、可靠性和效率：**Pass@1**表示首次尝试即可生成有效代码的案例百分比，反映生成准确率；**AVG Re**（平均重试次数）表示迭代修正循环中所需的平均重试次数，反映收敛效率。**SUC**表示成功生成语法正确CAD模型的案例比例，即能够无错误执行和渲染的模型。需注意，语法正确的模型未必与输入描述在语义上一致。**Tokens**测量生成单个CAD模型所需的平均标记数量。**Latency**则衡量生成单个CAD模型所需的平均时间。

表 2
针对200个测试模型开展的 ECIP 设计组件消融研究。

	密码：1↑	AVG Re↓	SUC↑
ECIP	**0.45**	**1.86**	**100.0%**
ECIP w/o Err	0.41	2.62	81.5%
无类型 ECIP	0.32	2.30	90.5%

5.4. 全面消融研究

为全面验证CADDesigner智能体中的设计选择及各组件的有效性，我们基于第5.2节所述的200个模型测试集开展了四组消融实验：第一组聚焦 ECIP 设计；第二组评估CAD工具与输入方式的影响；第三组分析不同大语言模型（LLM）后端的效果；第四组探究模型复杂度对标记使用频率及生成延迟的影响。除非另有说明，所有实验均采用纯文本输入。

5.4.1. ECIP 设计组件的影响

为阐明 ECIP 设计选择的优势（包括对大型语言模型友好的错误处理机制以及 ECIP 系统的整体架构），我们评估了三种 ECIP 变体：完整版 ECIP 、未配备详细错误处理系统的 ECIP 版本，以及未包含返回类型注释的 ECIP 版本。

表2的结果表明，在所有变体中，完整的 ECIP 配置实现了最佳的整体性能。当移除错误处理机制（ECIP 无错误处理）时，Pass@1从0.45略微下降至0.41，AVG Re从1.86升至2.62，而SUC则降至81.5%。这表明尽管智能体在初始阶段仍能生成正确的代码，但缺乏详细的反馈会减缓迭代修正过程中的收敛速度，并降低整体成功率。相比之下，移除类型注解（ECIP 无类型注解）导致Pass@1从0.45显著下降至0.32，说明在没有显式类型指导的情况下，初始代码生成更容易出现语义错误；AVG Re（2.30）虽高于完整 ECIP 方案，但仍低于无错误处理的 ECIP 方案，SUC也降至90.5%。这表明类型注解有助于将关键建模意图编码到提示中，但部分类型错误仍可通过重试进行修正。总体而言，我们观察到角色划分十分明确：错误处理机制有助于智能体在迭代过程中快速纠正错误；而类型标注则通过减少语义歧义来提升首次尝试的准确率。

5.4.2. CAD工具及输入方式的影响

为深入分析CADDesigner中各关键组件的具体贡献，我们评估了七个标记为A至G的变体，分别对应以下情况：(A) 从知识库中移除功能注释Anno；(B) 移除基础模型案例Case；(C) 禁用需求细化工具T_1；(D) 禁用可视化反馈工具T_4；(E) 全部采用纯文本输入的CADDesigner流程；(F) 全部采用纯图像输入的流程；(G) 全部采用文本与图像结合输入的流程。除变体F和G外，其余所有变体均仅使用文本输入。

表3汇总了多项评估指标的定量结果。在模型A中，移除语义API注释（Anno）后

Table 3
Ablation study of CADDesigner components and input modalities on 200 test models.

	IoU↑	CD↓	HD↓
A (w/o Anno)	–	–	–
B (w/o Case)	0.2650	0.1350	0.4690
C (w/o T_1)	0.2726	0.1251	0.5595
D (w/o T_4)	0.2522	0.1192	0.4793
E (Text only)	0.3041	0.1236	0.4154
F (Image only)	0.3518	0.1167	0.4262
G (Text + Image)	**0.3893**	**0.0693**	**0.2553**

Table 4
Ablation study of different LLM backends on 200 test models for ECIP code generation.

LLM Backend	IoU↑	CD↓	HD↓
Claude-4-Sonnet	**0.3041**	**0.1236**	**0.4154**
Gemini-2.5-Pro	0.2951	0.1571	0.4902
GPT-5.2-Codex	0.2914	0.1604	0.5021

prevents the agent from reliably identifying available operations and their usage, leading to failure to generate valid models. This indicates that structured function-level annotations play a critical role in grounding agent-based CAD code generation and guiding convergence toward correct solutions.

Models B, C, and D disable key system components—case examples (Case), requirement refinement (T_1), and visual feedback (T_4), respectively. Compared to the text-only variant (Model E), all three models show degraded performance in terms of IoU (0.2650, 0.2726, 0.2522 vs. 0.3041), suggesting that each component contributes positively to robust and accurate model generation. Specifically, removing structured case examples and requirement refinement increases semantic ambiguity during early-stage generation, while disabling visual feedback leads to the most significant drop in IoU, highlighting its critical role in iterative refinement and overall structural correctness.

Switching the input modality from text-only (Model E) to image-only (Model F) yields a notable performance improvement, suggesting that abstract textual descriptions alone are often insufficient to fully specify geometric details, whereas images provide richer spatial cues for CAD modeling. Model G, which combines both text and image as input, achieves the best results across all metrics, illustrating that textual information can effectively complement visual context by resolving ambiguities difficult to infer from images alone.

Overall, these results indicate that structured API knowledge, coordinated tool usage, and multimodal input jointly contribute to high-fidelity and robust CAD model generation.

5.4.3. Effect of LLM backend

To investigate the influence of different LLM backends on CAD code generation, we conduct an ablation study by replacing the LLM backend used in the code generation tool (T_2) of CADDesigner with three representative models: Claude-4-Sonnet, Gemini-2.5-Pro, and GPT-5.2-Codex. Each backend is evaluated under the ECIP code paradigm on 200 test models.

The quantitative results are reported in Table 4. We observe that CADDesigner achieves consistent performance across the three LLM backends evaluated. Claude-4-Sonnet attains the highest geometric fidelity, reflected by slightly higher IoU and lower CD and HD values. Gemini-2.5-Pro and GPT-5.2-Codex show comparable results, with small differences in CD and HD relative to Claude-4-Sonnet. These observations suggest that, among LLMs of similar capability, CADDesigner produces stable and reliable CAD code. The ECIP representation explicitly manages intermediate modeling states, and the knowledge-constrained generation framework provides structured guidance and reference cases, helping the code generation tool (T_2) produce scripts that are structurally correct and semantically meaningful. Overall, these results suggest that CADDesigner maintains broadly consistent CAD code generation across these backends, with moderate variations attributable to the underlying LLM.

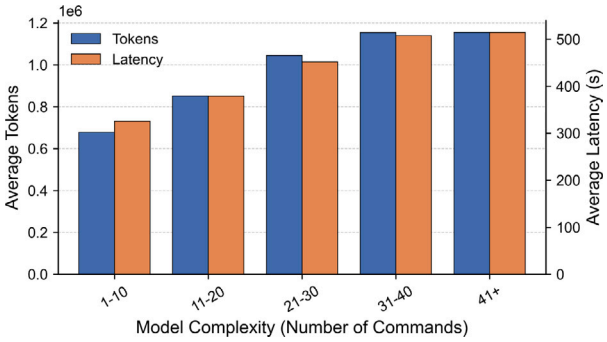

Fig. 4. Token usage and generation latency as a function of model complexity (number of commands). Models are grouped into bins of 10 commands each.

5.4.4. Effect of model complexity on inference cost

We evaluate the impact of model complexity on inference cost in CADDesigner. Specifically, we focus on two main metrics: Tokens and Latency. For memory consumption, CADDesigner primarily relies on external LLM APIs, resulting in negligible GPU memory overhead, while the main memory usage comes from RAG components (e.g., RAGFlow), which remain relatively stable and do not scale significantly with model complexity. Models are grouped into bins with an interval of 10 commands. For each complexity bin, we report the average Tokens and average Latency over all models within the bin. As shown in Fig. 4, both Tokens and Latency increase as model complexity grows. This is expected, since more complex models require the agent to generate longer CAD operation sequences and perform more reasoning steps during code generation. However, the increase is relatively moderate rather than steep. From 1–10 to 41 or more commands, token usage and latency grow steadily without abrupt escalation, indicating that the system does not incur excessive overhead as complexity increases. This behavior can be attributed to the design of CADDesigner. The explicit context representation in ECIP and the structured code generation process in T_2 help maintain stable reasoning and avoid redundant or excessively long intermediate steps. As a result, the inference cost scales in a controlled manner with model complexity.

We further examine the average contribution of each component in the pipeline across all 200 test models. Fig. 5 shows the average proportion of Tokens and Latency for the primary agent and the CAD tools. The code generation tool T_2 accounts for the largest share in both Tokens and Latency. The Requirement Refinement Tool T_1 and the Visual Feedback Tool T_4 each contribute relatively small portions, while the Command Execution Tool T_3 contributes modestly to latency without consuming tokens. The primary agent itself has only a minor contribution in both metrics. These observations indicate that T_2 is the main driver of both token and latency costs, whereas other components have limited impact on overall inference cost.

5.5. Comparison methods

We compare CADDesigner from two complementary perspectives: (1) different CAD representation paradigms used for code generation, and (2) existing text-to-CAD generation methods.

表 3

针对200个测试模型对CADDesigner组件及输入方式进行的消融研究。

	IoU↑	CD↓	HD↓
A（不含年份信息）	–	–	–
B（不含案例）	0.2650	0.1350	0.4690
C (w/o T_1)	0.2726	0.1251	0.5595
D (w/o T_4)	0.2522	0.1192	0.4793
E（仅文本）	0.3041	0.1236	0.4154
F（仅图像）	0.3518	0.1167	0.4262
G（文本 + 图像）	**0.3893**	**0.0693**	**0.2553**

表 4

针对200个测试模型，对不同LLM后端进行 ECIP 代码生成的消融研究。

LLM后端	IoU↑	CD↓	HD↓
克劳德四号十四行诗	**0.3041**	**0.1236**	**0.4154**
Gemini-2.5-Pro	0.2951	0.1571	0.4902
GPT-5.2-Codex	0.2914	0.1604	0.5021

这阻碍了智能体可靠地识别可用操作及其使用方式，导致无法生成有效的模型。这表明结构化的函数级注释在为基于智能体的CAD代码生成提供基础框架、并引导系统收敛于正确解决方案方面发挥着关键作用。

模型B、C和D分别禁用了关键系统组件——案例示例（Case）、需求细化（T_1）以及视觉反馈（T_4）。与纯文本版本（模型E）相比，这三个模型在IoU指标上的表现均有所下降（分别为0.2650、0.2726、0.2522对比0.3041），表明每个组件都对模型生成的稳健性和准确性具有积极贡献。具体而言，移除结构化案例示例和需求细化会增加早期生成阶段的语义歧义；而禁用视觉反馈则导致IoU指标出现最显著的下降，凸显了其在迭代优化及整体结构正确性中的关键作用。

将输入方式从纯文本（模型E）切换为纯图像（模型F）后，性能显著提升，这表明仅依靠抽象文本描述往往难以完整呈现几何细节，而图像能为CAD建模提供更丰富的空间信息。结合文本与图像作为输入的模型G在所有评估指标上均表现最佳，证明文本信息能够有效补充视觉上下文，解决仅凭图像难以判断的模糊问题。

总体而言，这些结果表明：结构化的API知识、协调的工具使用以及多模态输入共同促进了高保真度且稳健的CAD模型生成。

5.4.3. LLM后端的影响

为探究不同大语言模型（LLM）后端对CAD代码生成的影响，我们通过替换CADDesigner代码生成工具（T_2）中使用的LLM后端，采用三种代表性模型——Claude-4-Sonnet、Gemini-2.5-Pro和GPT-5.2-Codex——开展消融实验。每种后端均在 ECIP 代码范式下对200个测试模型进行评估。

定量结果详见表4。我们观察到，在评估的三种大语言模型（LLM）后端中，CADDesigner均展现出稳定的性能表现。Claude-4-Sonnet实现了最高的几何保真度，这体现在其交并比（IoU）略高、而一致性系数（CD）和高清指数（HD）较低；Gemini-2.5-Pro与GPT-5.2-Codex则呈现相近的结果，其CD和HD值与Claude-4-Sonnet存在微小差异。这些结果表明，在能力相近的大语言模型中，CADDesigner能够生成稳定且可靠的CAD代码。 ECIP 表示法明确管理中间建模状态，而知识约束生成框架提供了结构化指导和参考案例，有助于代码生成工具（T_2）生成结构正确且语义合理的脚本。总体而言，这些结果表明CADDesigner在不同后端上都能保持高度一致的CAD代码生成效果，其中适度差异主要源于底层大语言模型的不同。

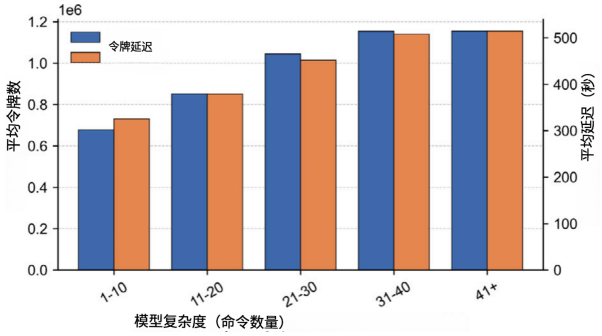

图4.令牌使用量与生成延迟随模型复杂度（命令数量）变化的关系。模型按每组10个命令进行分组。

5.4.4. 模型复杂度对推理成本的影响

我们评估了模型复杂度对CADDesigner推理成本的影响。具体而言，我们重点关注两个主要指标：令牌数和延迟时间。在内存消耗方面，CADDesigner主要依赖外部大语言模型API，因此GPU内存开销可忽略不计；而主内存使用量则来自RAG组件（如RAGFlow），这些组件的内存需求相对稳定，且不会随模型复杂度显著增加。我们将模型按每10条指令为一个区间进行分组，针对每个复杂度区间，我们统计了该区间内所有模型的平均令牌数和平均延迟时间。如图4所示，随着模型复杂度的提升，令牌数和延迟时间均呈上升趋势。这符合预期——更复杂的模型需要智能体生成更长的CAD操作序列并在代码生成过程中执行更多推理步骤。然而这种增长较为平缓而非急剧：从1-10条指令到41条或更多指令时，令牌使用量和延迟时间仅稳步上升而无突增现象，表明系统并未因复杂度增加而产生过度开销。这一特性可归因于CADDesigner的设计架构。ECIP 中的显式上下文表示与T_2中的结构化代码生成流程有助于保持推理稳定性，并避免冗余或过长的中间步骤。因此，推理成本随模型复杂度的变化呈现可控的增长趋势。

我们进一步分析了整个流程中各组件在全部200个测试模型中的平均贡献度。图5展示了主代理与CAD工具在令牌消耗量和延迟方面的平均占比。代码生成工具T_2在令牌消耗量和延迟方面均占据最大份额；需求细化工具T_1与可视化反馈工具T_4各自贡献较小；而命令执行工具T_3对延迟的影响较为有限且不消耗令牌；主代理本身在这两项指标上的贡献也微乎其微。这些结果表明：T_2是导致令牌消耗量和延迟成本的主要因素，其他组件对整体推理成本的影响则较为有限。

5.5. 比较方法

我们从两个互补的角度对CADDesigner进行比较：

(1) 用于代码生成的不同CAD表示范式，以及(2)现有的文本转CAD生成方法。

Table 5
Comparison of different CAD representation paradigms on 200 test models.

Representation	IoU↑	CD↓	HD↓	Pass@1↑	AVG Re↓	SUC↑	Tokens (M)↓	Latency (s)↓
ECIP	**0.3041**	**0.1236**	**0.4154**	0.46	**1.86**	**100.0%**	**0.98**	436
CadQuery	0.2827	0.1818	0.5186	0.50	2.01	87.5%	1.40	541
build123d	0.2617	0.1570	0.5126	**0.59**	1.91	96.0%	1.35	**363**

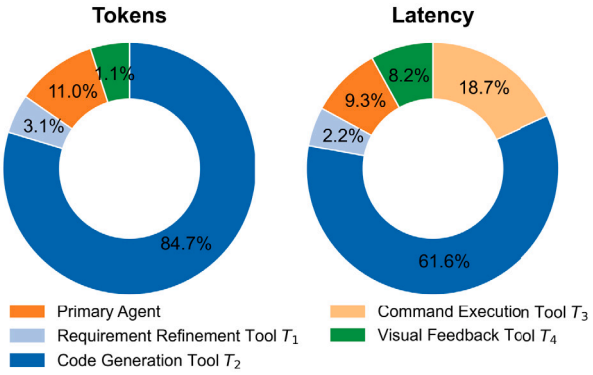

Fig. 5. Average inference cost breakdown across the CADDesigner pipeline. Tokens (left) and Latency (right) for each component.

5.5.1. Comparison of CAD representation paradigms

We first evaluate the impact of different CAD representation paradigms on code generation performance on 200 test models. Specifically, we compare our proposed ECIP paradigm with two widely used alternatives: CadQuery and build123d. To ensure a fair comparison, we integrate CadQuery and build123d into the same agent framework as CADDesigner by replacing the ECIP-based code generation component, while keeping all other components unchanged as described in Section 5.1. We further equip both CadQuery and build123d with RAG modules constructed from their official documentation. All methods use text-only input for evaluation. Here, ECIP, CadQuery, and build123d should be understood as different code representation styles or API interfaces used for generation, rather than fundamentally different geometric kernels.

The quantitative results presented in Table 5 illustrate clear differences among the evaluated CAD code representations. ECIP achieves the best IoU and SUC, as well as the lowest AVG Re, indicating stronger geometric fidelity and more reliable iterative convergence. Since ECIP is implemented as a lightweight representation layer on top of CadQuery, these differences should be understood primarily at the level of code representation and LLM usability rather than the underlying geometric engine. In particular, although CadQuery can also support coding patterns that partially overlap with ECIP, its commonly used fluent interface does not explicitly standardize intermediate modeling context for language models. By contrast, ECIP consistently organizes each operation into explicit object-and-parameter transformations, which makes dependency tracking, code generation, and iterative refinement more reliable for LLM-based agents. Build123d attains the highest Pass@1 and shortest latency, indicating that its compact syntax can often be quickly converted into executable code. However, its lower IoU and higher CD and HD suggest that executable code does not necessarily imply geometric accuracy, which may be due to its reliance on overloaded symbolic operators, making modeling semantics harder for LLMs to interpret than explicit function calls. Overall, the results highlight trade-offs among representation styles for LLM-oriented CAD coding: APIs that simplify LLM code generation can improve early success and reduce execution time, but may compromise geometric accuracy.

To further assess robustness and distribution-level differences among ECIP, CadQuery, and build123d, we analyze the distributions of IoU, CD, and HD across the 200 test models. As shown in Fig. 6, ECIP exhibits higher median IoU and lower CD/HD values with tighter distributions, indicating more consistent geometric fidelity. In comparison, CadQuery and build123d show wider variability and higher CD/HD, highlighting less precise and less reliable CAD model generation. These results suggest that ECIP demonstrates more accurate and consistent performance across the evaluated models.

Table 6
Performance comparison of different Text-to-CAD methods under abstract text inputs on $1K$ test models.

Method	IoU↑	CD↓	HD↓	SUC↑
Text2CAD	0.1831	0.1475	0.5680	96.6%
cadrille	0.0274	0.2162	0.5817	98.2%
CADCodeVerify	0.2348	0.2329	0.4892	86.1%
CADDesigner	**0.2769**	**0.1097**	**0.4347**	**100.0%**

5.5.2. Comparison with existing methods

We further compare our method with two learning-based text-to-CAD generation methods: Text2CAD [4], cadrille [28] and one agent-based method: CADCodeVerify [25].

- **Text2CAD** proposes an end-to-end transformer-based auto-regressive network to generate parametric CAD models from input texts.
- **cadrille** proposes a two-stage CAD reconstruction pipeline: supervised fine-tuning (SFT) on large-scale procedurally generated data, followed by reinforcement learning fine-tuning using online feedback.
- **CADCodeVerify** proposes an agent that utilizes validation questions and visual feedback for design verification.

For evaluation, we use the $1K$-model subset described in Section 5.2, randomly sampled from the de-duplicated DeepCAD test set. For each instance, we use the abstract text description as input. We then perform inference using their released weights and our method to generate results for metric calculation. As presented in Table 6 and Fig. 7, the experimental results show that CADDesigner achieves the best performance. CADDesigner demonstrates the best prompt-result alignment, followed by CADCodeVerify and Text2CAD. While cadrille achieves a high SUC score, indicating it can produce syntactically valid CAD models in most cases, the visualized results reveal these outputs are typically meaningless with respect to the input instructions (see Fig. 7). This failure is due to cadrille's poor generalization from expert-level to abstract instructions: it generates structurally valid but semantically irrelevant models. In contrast, the other methods generate outputs that are both syntactically and semantically valid to varying degrees, with our method being the most consistent.

Additionally, we compare our method with cadrille on image-based inputs to further illustrate their respective modeling capabilities. As shown in Fig. 8, CADDesigner consistently generates CAD models that closely match the input images, benefiting from a more expressive CAD operation space. In particular, several models require rotationally symmetric structures that are naturally constructed via revolve operations, which are explicitly supported by CADDesigner but not by cadrille, leading to missing or distorted geometries in the latter. Moreover, models with rich geometric details and repetitive structures are more effectively specified using high-level CAD operations such as arrays or patterned constructs. CADDesigner can leverage these higher-level APIs

表 5
基于200个测试模型对不同CAD表示范式进行比较。

表示	IoU↑	CD↓	HD↓	密码：1↑	AVG Re↓	SUC↑	代币数(M)↓	延迟（秒）↓
ECIP	**0.3041**	**0.1236**	**0.4154**	0.46	**1.86**	**100.0%**	**0.98**	436
CadQuery	0.2827	0.1818	0.5186	0.50	2.01	87.5%	1.40	541
build123d	0.2617	0.1570	0.5126	**0.59**	1.91	96.0%	1.35	**363**

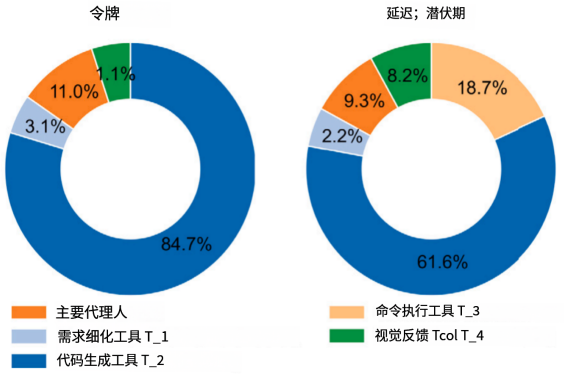

图5.CADDesigner流程中各环节的平均推理成本构成：各组件对应的令牌消耗量（左）与延迟值（右）。

5.5.1. CAD表示范式的比较

我们首先在200个测试模型上评估了不同CAD表示范式对代码生成性能的影响。具体而言，我们将提出的 ECIP 范式与两种广泛使用的替代方案——CadQuery 和 build123d 进行对比。为确保公平比较，我们将CadQuery和build123d整合到与CADDesigner相同的代理框架中：通过替换基于 ECIP 的代码生成组件，同时保持其他所有组件不变（详见第5.1节）。此外，我们还为CadQuery和build123d配备了根据其官方文档构建的RAG模块。所有方法均采用纯文本输入进行评估。需要明确的是，ECIP 、CadQuery和build123d应被视为用于代码生成的不同代码表示风格或API接口，而非本质不同的几何核心算法。

表5中呈现的定量结果清晰展示了不同CAD代码表示方法之间的差异。 ECIP 在交并比（IoU）和相似度评分（SUC）方面表现最佳，同时平均重合率（AVG Re）最低，表明其具有更强的几何保真度和更可靠的迭代收敛性。由于 ECIP 是作为基于CadQuery构建的轻量级表示层实现的，这些差异主要体现在代码表示层面和大语言模型的使用体验上，而非底层几何引擎本身。值得注意的是，尽管CadQuery也能支持与 ECIP 部分重叠的编码模式，但其常用的流畅接口并未明确为语言模型标准化中间建模上下文。相比之下， ECIP 始终将每个操作步骤明确定义为对象与参数转换，这使得基于大语言模型的智能体在依赖关系追踪、代码生成及迭代优化方面更具可靠性。Build123d则实现了最高的Pass@1值和最短的延迟时间，说明其紧凑的语法结构通常能快速转化为可执行代码。然而，其较低的IoU值以及较高的CD和HD指标表明，可执行代码并不必然保证几何精度——这可能源于其对重载符号运算符的依赖，使得大型语言模型（LLM）更难理解建模语义，相较于显式函数调用而言。总体而言，研究结果凸显了面向LLM的CAD编码中不同表示方式之间的权衡关系：能够简化LLM代码生成的API虽可提升早期成功率并缩短执行时间，但可能会牺牲几何精度。

为进一步评估 ECIP 、CadQuery和build123d在稳健性及分布层面的差异，我们分析了其分布情况。

表 6
在1000个测试模型上，针对抽象文本输入，不同文本转CAD方法的性能比较。

方法	IoU↑	CD↓	HD↓	SUC↑
Text2CAD	0.1831	0.1475	0.5680	96.6%
cadrille	0.0274	0.2162	0.5817	98.2%
CAD代码验证	0.2348	0.2329	0.4892	86.1%
CAD设计师	**0.2769**	**0.1097**	**0.4347**	**100.0%**

在200个测试模型中，IoU、CD和HD指标的表现如下。图6显示， ECIP 具有更高的中位数IoU值、更低的CD/HD值且分布更集中，表明其几何保真度更为一致；相比之下，CadQuery和build123d的变异范围更大、CD/HD值更高，说明其CAD模型生成精度和可靠性较低。这些结果表明，在所有评估模型中， ECIP 均展现出更准确且一致的性能表现。

5.5.2. 与现有方法的比较

我们进一步将我们的方法与两种基于学习的文本到CAD生成方法进行了比较：Text2CAD[4]、cadrille[28]以及一种智能体方法。
基于方法：CADCodeVerify[25]。

- **Text2CAD**提出了一种基于Transformer的端到端自回归网络，能够从输入文本生成参数化CAD模型。
- **cadrille**提出了一种两阶段CAD重建流程：首先在大规模程序生成数据上进行监督微调（SFT），随后利用在线反馈进行强化学习微调。
- **CADCodeVerify**提出了一种利用验证问题和视觉反馈进行设计验证的智能代理。

在评估阶段，我们采用第5.2节所述的1k模型子集，该子集是从去重后的DeepCAD测试集中随机抽取的。对于每个实例，我们以抽象文本描述作为输入，随后利用其公开的权重参数及我们的方法进行推理，生成用于指标计算的结果。如表6和图7所示，实验结果表明CADDesigner实现了最佳性能：其提示-结果一致性最为出色，其次为CADCodeVerify和Text2CAD。虽然cadrille获得了较高的SUC分数（表明其在多数情况下能生成语法正确的CAD模型），但可视化结果显示这些输出通常与输入指令缺乏关联性（参见图7）。这一缺陷源于cadrille在从专家级指令向抽象指令泛化时能力不足——它生成的模型结构正确但语义无关。相比之下，其他方法生成的输出在语法和语义层面均具有不同程度的有效性，而我们的方法表现最为稳定。

此外，我们还将我们的方法与cadrille在基于图像的输入场景下的表现进行对比，以进一步展示两者各自的建模能力。如图8所示，CADDesigner始终能生成与输入图像高度匹配的CAD模型，并充分利用了更丰富的CAD操作空间。特别是，许多模型需要具有旋转对称结构——这类结构可通过旋转操作自然构建，而CADDesigner明确支持该功能，但cadrille则无法实现，导致后者的几何形状出现缺失或畸变。此外，对于包含丰富几何细节和重复结构的模型，使用数组或图案化构造等高级CAD操作能更有效地进行建模。相较于其他方法，CADDesigner能够充分

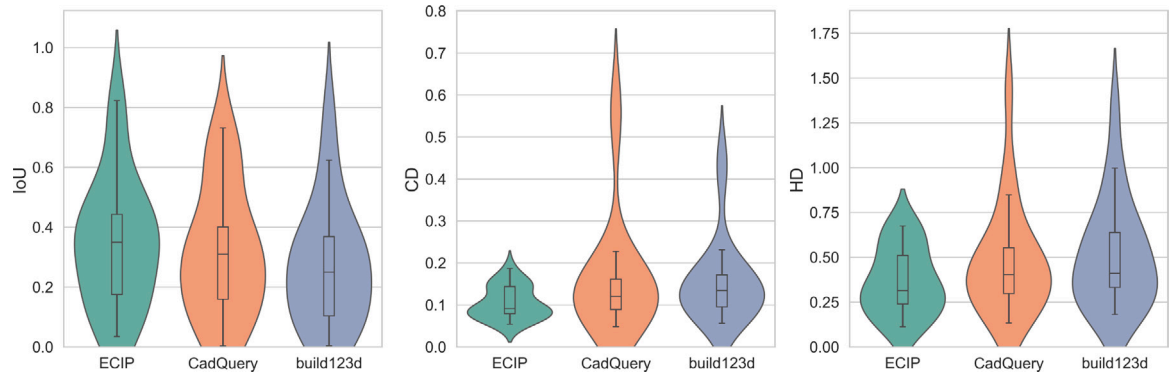

Fig. 6. Violin plots comparing the distributions of IoU, CD, and HD across ECIP, CadQuery, and build123d on 200 test models. ECIP achieves higher geometric fidelity and more consistent performance compared to the other paradigms.

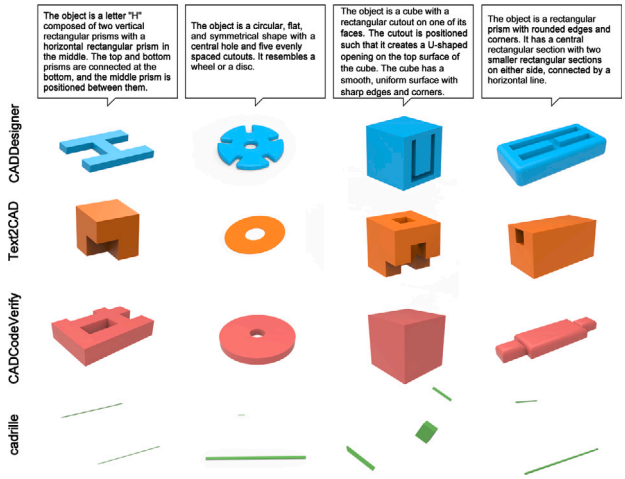

Fig. 7. Comparison of generation results across Text2CAD, CADCodeVerify, cadrille, and our method. CADDesigner achieves the best input alignment. Text2CAD and CADCodeVerify show moderate performance, while cadrille generates syntactically valid but semantically incorrect outputs due to poor generalization from expert-level training to abstract inputs.

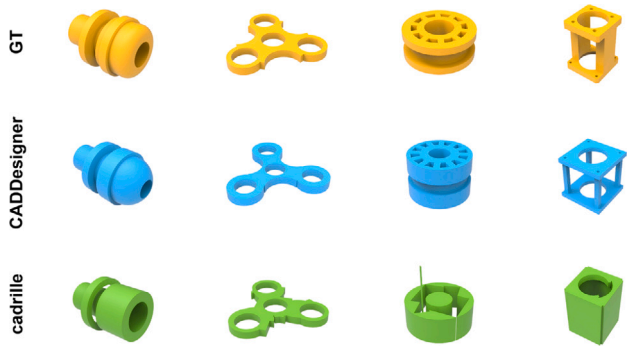

Fig. 8. Comparison of CADDesigner and cadrille on image-based inputs. CADDesigner (blue) generates CAD models that more closely match the input images on these representative conceptual cases, benefiting from support for operations such as revolve and pattern-based constructions. In contrast, cadrille (green), relying on low-level extrusion, often produces geometrically less faithful results.

to compactly and precisely generate such structures, whereas cadrille relies primarily on low-level extrusion-based operations, often resulting in geometrically less faithful outputs.

These results highlight that beyond syntactic validity, the availability of expressive and semantically aligned CAD operations is crucial for faithful image-based CAD model generation.

5.6. Visual results presentation

We additionally provide qualitative results generated from sketch–text inputs, with representative examples illustrated in Fig. 9. The content presented here is derived from the DeepCAD dataset, the CADCodeVerify dataset, and datasets covering a wide range of geometries, including standard industrial components (e.g., bolts, flanges, and pipe fittings), as well as more complex models and assemblies.

5.7. Limitations

Although the latency of CADDesigner scales in a relatively controlled manner with increasing model complexity, its end-to-end response time remains non-negligible in interactive usage scenarios, particularly for complex model generation and multi-round iterative refinement. Our analysis as shown in Fig. 5 reveals that the code generation module (T_2) is the primary contributor to the overall system latency. Therefore, improving inference efficiency remains a critical challenge.

Furthermore, the current framework does not yet sufficiently incorporate the domain-specific knowledge required in industrial design practice, such as manufacturing constraints, tolerance requirements, assembly semantics, and standardized engineering conventions. Therefore, the capability for complex model generation and validation in specific industrial domains still requires further improvement.

6. Conclusion and future work

We propose CADDesigner, a novel framework for CAD modeling code generation, combined with an explicit context imperative paradigm to produce high-quality CAD modeling scripts. Experimental results show that CADDesigner achieves competitive performance and outperforms the evaluated baselines in conceptual CAD design tasks. CADDesigner is particularly well-suited for rapid prototyping and early-stage conceptual CAD modeling, where user intent is often abstract and iterative refinement plays a critical role.

In future work, we plan to extend CADDesigner by incorporating learning-based methods that accept point clouds and B-rep data as input, allowing the system to learn geometric and topological constraints through data-driven training. Furthermore, we aim to further optimize the inference efficiency of the framework and introduce domain-specific industrial cases and functional constraints, thereby improving the generation capability and efficiency of our method for complex models in industrial scenarios.

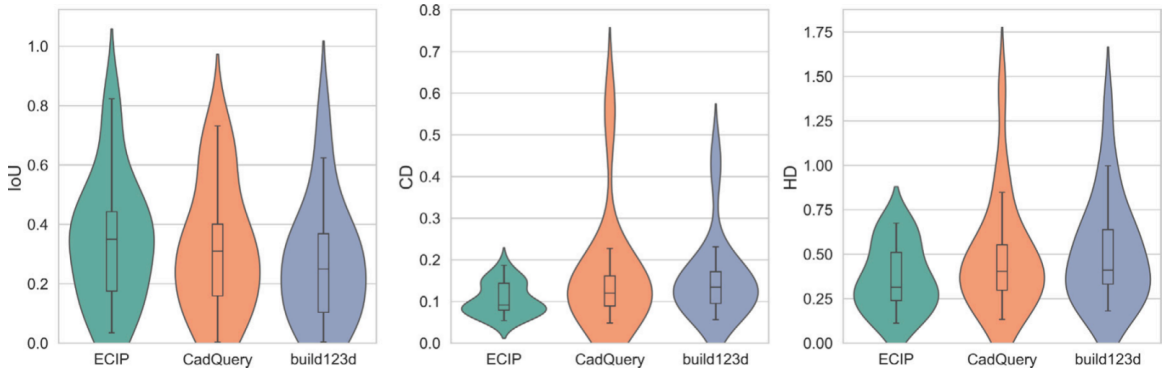

图6.小提琴图对比了 ECIP 、CadQuery和build123d在200个测试模型上IoU、CD及HD指标的分布情况。 ECIP 实现了更高的几何匹配度。

利用这些高级API，从而获得更高的建模保真度和更一致的性能表现。

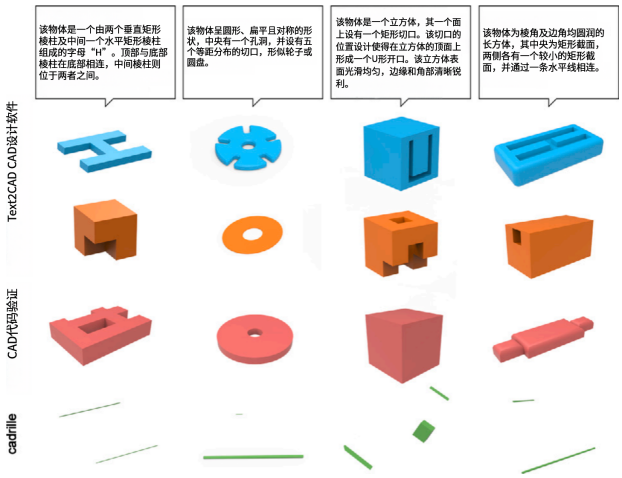

图 7. Text2CAD、CADCodeVerify、cadrille 与我们方法的生成结果对比。CADDesigner实现了最佳的输入对齐效果；Text2CAD和CADCodeVerify表现中等；而cadrille由于从专家级训练向抽象输入的泛化能力较差，生成的输出在语法上正确但语义不准确。

能够紧凑且精确地生成此类结构，而cadrille则主要依赖基于低级挤出操作的方法，往往导致输出结果的几何精度较低。

这些结果表明，除语法有效性外，具备表达性且语义一致的CAD操作对于实现精准的基于图像的CAD模型生成至关重要。

5.6. 可视化结果展示

我们还提供了基于草图-文本输入生成的定性结果，代表性示例如图9所示。本文呈现的内容源自DeepCAD数据集、CADCodeVerify数据集以及涵盖多种几何形状的数据集（包括标准工业部件（如螺栓、法兰和管件）以及更复杂的模型与装配体）。

5.7. 局限性

尽管CADDesigner的延迟随模型复杂度增加而呈现相对可控的增长趋势，但在交互式使用场景中（尤其是进行复杂模型生成和多轮迭代优化时），其端到端响应时间仍不容忽视。如图5所示分析结果表明，代码生成模块（T_2）是导致系统整体延迟的主要因素。因此，提升推理效率

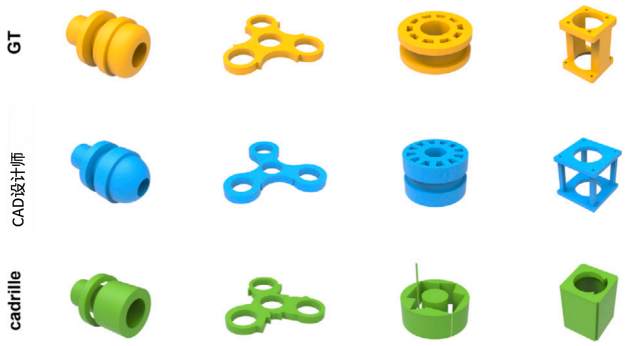

图8.CADDesigner与cadrille在基于图像输入情况下的性能对比。在这些代表性概念案例中，CADDesigner（蓝色）生成的CAD模型与输入图像更为吻合，这得益于其对旋转操作及基于图案构建等功能的支持；而cadrille（绿色）则依赖底层挤出算法，往往会产生几何精度较低的结果。

仍是亟待解决的关键问题。

此外，当前框架尚未充分融入工业设计实践中所需的领域特定知识，例如制造约束条件、公差要求、装配语义以及标准化工程规范。因此，在特定工业领域中实现复杂模型的生成与验证功能仍需进一步完善。

6. 结论与未来研究方向

我们提出CADDesigner——一种创新的CAD建模代码生成框架，结合显式上下文指令范式，可生成高质量的CAD建模脚本。实验结果表明，在概念性CAD设计任务中，CADDesigner展现出优异性能，超越了所有评估基准方法。该框架特别适用于快速原型开发及早期概念性CAD建模场景，这类场景中用户需求往往较为抽象，迭代优化过程至关重要。

在后续工作中，我们计划通过引入基于学习的方法（这些方法可接受点云和B-rep数据作为输入），对CADDesigner进行扩展，使系统能够通过数据驱动的训练学习几何与拓扑约束。此外，我们还致力于进一步优化该框架的推理效率，并引入特定领域的工业应用场景及功能约束条件，从而提升本方法在工业场景中处理复杂模型时的生成能力与运行效率。

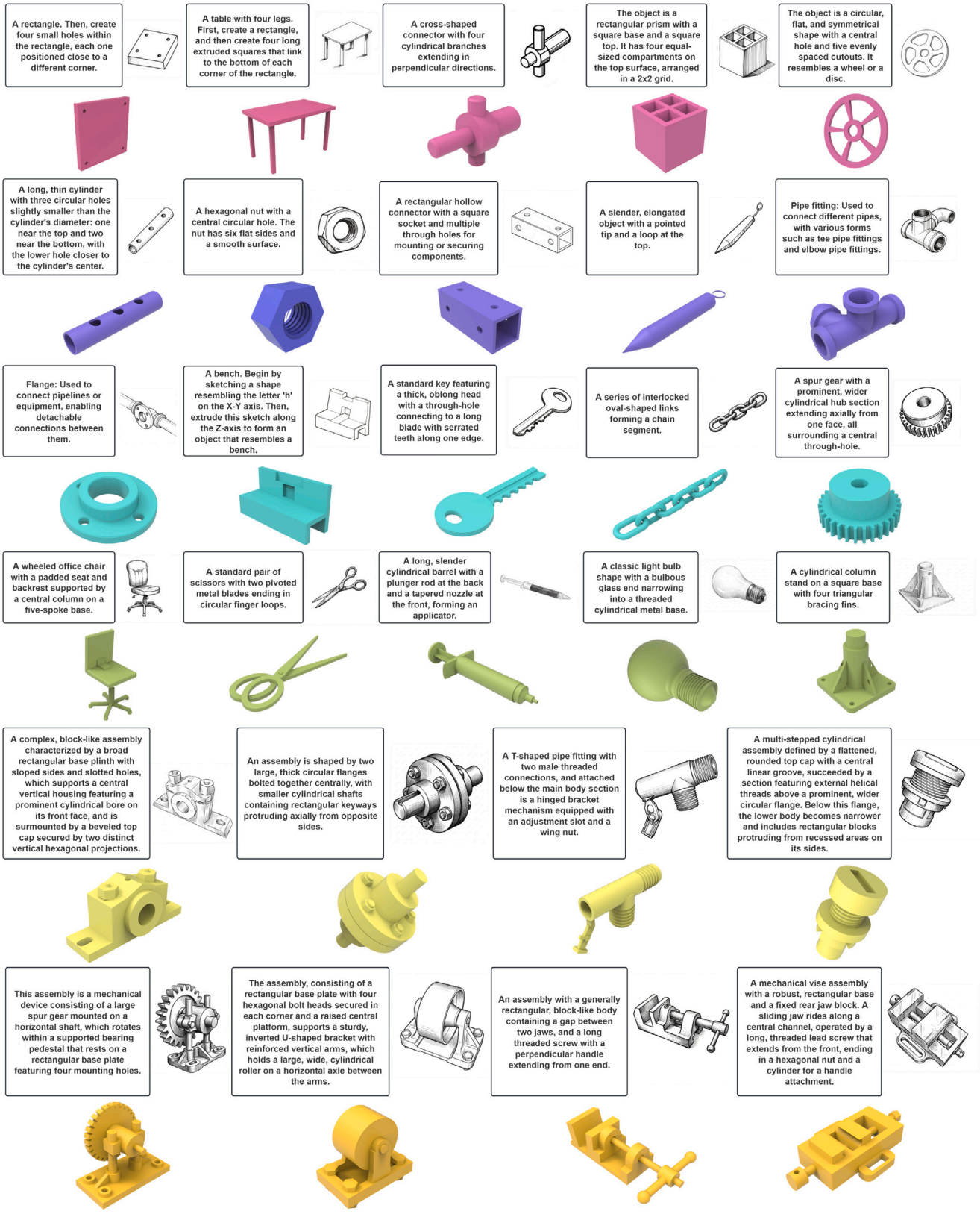

Fig. 9. Visual results with sketch–text input.

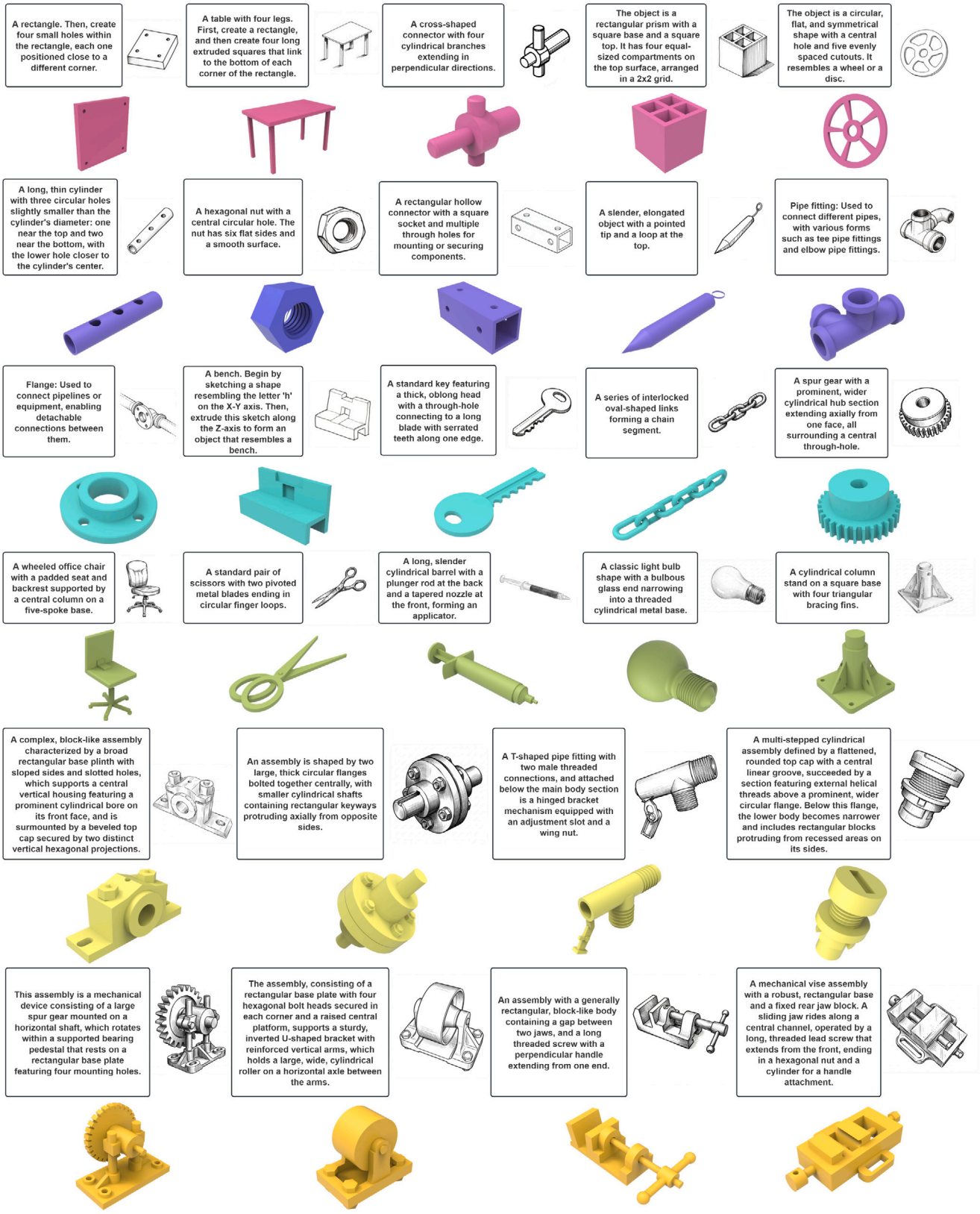

图 9.使用草图-文本输入方式的可视化结果。

CRediT authorship contribution statement

Fengxiao Fan: Writing – review & editing, Writing – original draft, Software. **Jingzhe Ni:** Writing – review & editing, Writing – original draft, Software. **Xiaolong Yin:** Writing – original draft. **Sirui Wang:** Resources. **Xingyu Lu:** Writing – original draft, Software. **Qiang Zou:** Conceptualization. **Ruofeng Tong:** Conceptualization. **Min Tang:** Conceptualization. **Peng Du:** Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Leading Goose R & D Program of Zhejiang under Grant No. 2024C01103.

Data availability

Data will be made available on request.

References

[1] Wu Rundi, Xiao Chang, Zheng Changxi. DeepCAD: A deep generative network for computer-aided design models. In: Proceedings of the IEEE/CVF international conference on computer vision. 2021, p. 6772–82.
[2] Li Xueyang, Lou Yunzhong, Song Yu, Zhou Xiangdong. Mamba-CAD: state space model for 3D computer-aided design generative modeling. In: Proceedings of the AAAI conference on artificial intelligence, vol. 39, 2025, p. 5013–21.
[3] Khan Mohammad Sadil, Dupont Elona, Ali Sk Aziz, Cherenkova Kseniya, Kacem Anis, Aouada Djamila. CAD-SIGNet: CAD language inference from point clouds using layer-wise sketch instance guided attention. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2024, p. 4713–22.
[4] Khan Mohammad Sadil, Sinha Sankalp, Sheikh Talha Uddin, Stricker Didier, Ali Sk Aziz, Afzal Muhammad Zeshan. Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts. In: Advances in neural information processing systems, vol. 37, 2024, p. 7552–79.
[5] Li Jiahao, Ma Weijian, Li Xueyang, Lou Yunzhong, Zhou Guichun, Zhou Xiangdong. CAD-Llama: Leveraging large language models for computer-aided design parametric 3D model generation. In: Proceedings of the computer vision and pattern recognition conference. 2025, p. 18563–73.
[6] Wang Siyu, Chen Cailian, Le Xinyi, Xu Qimin, Xu Lei, Zhang Yanzhou, Yang Jie. CAD-GPT: synthesising CAD construction sequence with spatial reasoning-enhanced multimodal LLMs. In: Proceedings of the AAAI conference on artificial intelligence, vol. 39, 2025, p. 7880–8.
[7] Rukhovich Danila, Dupont Elona, Mallis Dimitrios, Cherenkova Kseniya, Kacem Anis, Aouada Djamila. CAD-Recode: Reverse engineering CAD code from point clouds. 2024, arXiv preprint arXiv:2412.14042.
[8] CadQuery contributors. CadQuery. 2026.
[9] Willis Karl DD, Pu Yewen, Luo Jieliang, Chu Hang, Du Tao, Lambourne Joseph G, Solar-Lezama Armando, Matusik Wojciech. Fusion 360 gallery: A dataset and environment for programmatic CAD construction from human design sequences. ACM Trans Graph 2021;40(4).
[10] Xu Xiang, Willis Karl DD, Lambourne Joseph G, Cheng Chin-Yi, Jayaraman Pradeep Kumar, Furukawa Yasutaka. SkexGen: autoregressive generation of CAD construction sequences with disentangled codebooks. In: International conference on machine learning. 2022, p. 24698–724.
[11] Zhang Aijia, Jia Weiqiang, Zou Qiang, Feng Yixiong, Wei Xiaoxiang, Zhang Ye. Diffusion-CAD: Controllable diffusion model for generating computer-aided design models. IEEE Trans Vis Comput Graphics 2025.
[12] Guo Haoxiang, Liu Shilin, Pan Hao, Liu Yang, Tong Xin, Guo Baining. Complex-Gen: CAD reconstruction by B-rep chain complex generation. ACM Trans Graph 2022;41(4):1–18.
[13] Ma Weijian, Chen Shuaiqi, Lou Yunzhong, Li Xueyang, Zhou Xiangdong. Draw step by step: reconstructing CAD construction sequences from point clouds via multimodal diffusion. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2024, p. 27154–63.
[14] Li Yuan, Lin Cheng, Liu Yuan, Long Xiaoxiao, Zhang Chenxu, Wang Ningna, Li Xin, Wang Wenping, Guo Xiaohu. CADDreamer: CAD object generation from single-view images. In: Proceedings of the computer vision and pattern recognition conference. 2025, p. 21448–57.
[15] Chen Cheng, Wei Jiacheng, Chen Tianrun, Zhang Chi, Yang Xiaofeng, Zhang Shangzhan, Yang Bingchen, Foo Chuan-Sheng, Lin Guosheng, Huang Qixing, Liu Fayao. CADCrafter: Generating computer-aided design models from unconstrained images. In: Proceedings of the computer vision and pattern recognition conference. 2025, p. 11073–82.
[16] Wu Sifan, Khasahmadi Amir, Katz Mor, Jayaraman Pradeep Kumar, Pu Yewen, Willis Karl, Liu Bang. CAD-LLM: large language model for cad generation. In: Proceedings of the neural information processing systems conference. 2023.
[17] Wu Sifan, Khasahmadi Amir Hosein, Katz Mor, Jayaraman Pradeep Kumar, Pu Yewen, Willis Karl, Liu Bang. CadVLM: Bridging language and vision in the generation of parametric CAD sketches. In: European conference on computer vision. 2024, p. 368–84.
[18] Xu Jingwei, Zhao Zibo, Wang Chenyu, Liu Wen, Ma Yi, Gao Shenghua. CAD-MLLM: Unifying multimodality-conditioned CAD generation with MLLM. 2024, arXiv preprint arXiv:2411.04954.
[19] Qwen Team. Qwen2 technical report. 2024, arXiv preprint arXiv:2407.10671.
[20] Xie Haoyang, Ju Feng. Text-to-CadQuery: A new paradigm for CAD generation with scalable large model capabilities. 2025, arXiv preprint arXiv:2505.06507.
[21] Mallis Dimitrios, Karadeniz Ahmet Serdar, Cavada Sebastian, Rukhovich Danila, Foteinopoulou Niki, Cherenkova Kseniya, Kacem Anis, Aouada Djamila. CAD-Assistant: Tool-augmented VLLMs as generic CAD task solvers. 2024, arXiv preprint arXiv:2412.13810.
[22] Riegel Juergen, Mayer Werner, van Havre Yorik. FreeCAD: open-source parametric 3D CAD modeler. 2024.
[23] Li Xueyang, Li Jiahao, Song Yu, Lou Yunzhong, Zhou Xiangdong. Seek-CAD: A self-refined generative modeling for 3D parametric CAD using local inference via DeepSeek. 2025, arXiv preprint arXiv:2505.17702.
[24] Yuan Zeqing, Lan Haoxuan, Zou Qiang, Zhao Junbo. 3D-PreMise: Can large language models generate 3D shapes with sharp features and parametric control? 2024, arXiv preprint arXiv:2401.06437.
[25] Alrashedy Kamel, Tambwekar Pradyumna, Zaidi Zulfiqar, Langwasser Megan, Xu Wei, Gombolay Matthew. Generating CAD code with vision-language models for 3d designs. 2024, arXiv preprint arXiv:2410.05340.
[26] Maitland Roger, jdegenstein, Bernhard, Rooke Ethan, Mobley JR, snoyer, Jojain, Häberle Andreaz Felix, Ruud, Fischman Ami, McMullan Jason S, Dvořák Roman, simon klemenc, BogdanTheGeek, Spectre5, Frívaldský Dalibor, D'Orazio Daniele, George, hoijui, OpenVMP, Yeicor, Steppke Alexander, mayhem 64, luzpaz, nobkd, Poughon Victor, slobberingant, Bosch Arno, Walters Barnaby, Eiden Matti. Gumyr/build123d: v0.9.1. 2025.
[27] Yao Shunyu, Zhao Jeffrey, Yu Dian, Du Nan, Shafran Izhak, Narasimhan Karthik R, Cao Yuan. ReAct: Synergizing reasoning and acting in language models. In: The eleventh international conference on learning representations. 2023.
[28] Kolodiazhnyi Maksim, Tarasov Denis, Zhemchuzhnikov Dmitrii, Nikulin Alexander, Zisman Ilya, Vorontsova Anna, Konushin Anton, Kurenkov Vladislav, Rukhovich Danila. cadrille: Multi-modal CAD reconstruction with online reinforcement learning. 2025, arXiv preprint arXiv:2505.22914.

CRediT作者贡献声明

范凤晓：写作——审阅与编辑；写作——初稿撰写；软件。**倪静哲：**写作——审阅与编辑；写作——初稿撰写；软件。**尹小龙：**写作——初稿撰写。**王思瑞：**资源。**卢星宇：**写作——初稿撰写；软件。**强邹：**概念化。**童若峰：**概念化。**唐敏：**概念化。**杜鹏：**概念化。

利益冲突声明

作者声明，其不存在任何已知的可能影响本文所述研究结果的竞争性经济利益或个人关系。

致谢

本研究由浙江省领军企业研发计划（项目编号：2024C01103）资助。

数据可用性

数据可根据要求提供。

参考文献

[1] 吴润迪、肖畅、郑昌熙。DeepCAD：一种用于计算机辅助设计模型的深度生成网络。收录于：IEEE/ CVF 国际计算机视觉会议论文集，2021年，第6772–82页。

[2] 李学阳、楼云中、宋宇、周向东。Mamba-CAD：用于三维计算机辅助设计生成建模的状态空间模型。载于：《AAAI人工智能会议论文集》，第39卷，2025年，第5013–21页。

[3] Khan Mohammad Sadil，Dupont Elona，Ali Sk Aziz，Cherenkova Kseniya，Kacem Anis，Aouada Djamila. CAD-SIGNet：基于逐层草图实例引导注意力机制从点云中推断CAD语言。收录于：IEEE/ CVF 计算机视觉与模式识别会议论文集，2024年，第4713–22页。

[4] 汗·穆罕默德·萨迪尔、辛哈·桑卡尔普、谢赫·塔尔哈·乌丁、斯特里克·迪迪埃、阿里·斯克·阿齐兹、阿夫扎勒·穆罕默德·泽尚。Text2CAD：根据从初学者到专家级别的文本提示生成顺序CAD设计。载于：《神经信息处理系统进展》第37卷，2024年，第7552–79页。

[5] 李佳浩、马伟健、李学阳、楼云中、周贵春、周向栋。CAD-Llama：利用大型语言模型进行计算机辅助设计参数化3D模型生成。载于：计算机视觉与图案识别会议论文集，2025年，第18563–1873页。

[6] 王思宇、陈彩莲、勒新毅、徐启民、徐磊、张彦洲、杨杰。CAD-GPT：基于空间推理的CAD构建序列合成——增强型多模态大语言模型。载于：《AAAI人工智能会议论文集》第39卷，2025年，第7880–7888页。

[7] 鲁赫维奇·达尼拉、杜邦·埃洛娜、马利斯·迪米特里奥斯、切连科娃·克谢尼娅、卡切姆·阿尼斯、奥瓦达·贾米拉。CAD-Recode：基于点云逆向工程生成CAD代码。2024年，arXiv预印本arXiv:2412.14042。

[8] CadQuery贡献者。CadQuery。2026年。

[9] Willis Karl DD、Pu Yewen、Luo Jieliang、Chu Hang、Du Tao、Lambourne Joseph G，Solar-Lezama Armando、Matusik Wojciech。Fusion 360图库：一个用于基于人工设计序列进行程序化CAD构建的数据集及环境。ACM Trans Graph 2021；40(4).

[10] 徐翔、Willis Karl DD、Lambourne Joseph G、Cheng Chin-Yi、Jayara-Man Pradeep Kumar、Furukawa Yasutaka。SkexGen：一种采用解耦码本的自回归生成CAD构造序列方法。收录于：国际机器学习会议，2022年，第24698–2724页。

[11] 张爱佳、贾伟强、邹强、冯一雄、魏晓翔、张业。Diffusion-CAD：用于生成计算机辅助设计模型的可控扩散模型。IEEE Transactions on Visual Computing Graphics 2025。

[12] 郭浩祥、刘世林、潘浩、刘阳、童欣、郭柏宁。复杂结构——Gen：基于B-Rep链复合体生成的CAD重构。ACM图论汇刊2022；41(4):1–18。

[13] 马伟健、陈帅奇、楼云中、李学阳、周向东。逐步绘制：通过多模态扩散从点云重建CAD构建序列。载于：IEEE/ CVF 计算机视觉与模式识别会议论文集，2024年，第27154–63页。

[14] 李元、林成、刘元、龙晓晓、张晨旭、王宁娜；李欣、王文平、郭晓虎。CADDreamer：基于单视图图像生成CAD对象。收录于：计算机视觉与模式识别会议论文集，2025年，第21448–57页。

[15] 陈诚、魏佳城、陈天润、张驰、杨晓峰、张尚湛、杨炳臣、傅传胜、林国胜、黄启景、刘发耀。CADCrafter：从无约束图像生成计算机辅助设计模型。载于：计算机视觉与模式识别会议论文集，2025年，第11073–1182页。

[16] 吴思凡、哈萨赫马迪·阿米尔、卡茨·莫尔、贾亚拉曼·普拉迪普·库马尔、普·叶文、威利斯·卡尔、刘邦。CAD-LLM：用于CAD生成的大语言模型。收录于：神经信息处理系统会议论文集，2023。

[17] 吴思凡、哈萨赫马迪·阿米尔·侯赛因、卡茨·莫尔、贾亚拉曼·普拉迪普·库马尔、普·叶文、威利斯·卡尔、刘邦。CadVLM：在参数化CAD草图生成中弥合语言与视觉的鸿沟。载于：欧洲计算机视觉会议，2024年，第368–384页。

[18] 徐景伟、赵子博、王晨宇、刘文、马毅、高胜华。cadmllm：将多模态条件CAD生成与 MLLM 技术相结合。2024年，arXiv预印本arXiv:2411.04954。

[19] Qwen Team . Qwen2技术报告。2024年，arXiv预印本arXiv:2407.10671。

[20] Xie Haoyang，Ju Feng . 文本到CAD查询：一种具备可扩展大型模型能力的CAD生成新范式。2025年，arXiv预印本arXiv:2505.06507。

[21] 马利斯·迪米特里奥斯、卡拉德尼兹·艾哈迈德·塞尔达尔、卡瓦达·塞巴斯蒂安、鲁科维奇·达尼拉、福泰诺普卢·妮基、切连科娃·克谢尼娅、卡切姆·阿尼斯、奥瓦达·贾米拉。CAD助理：基于工具增强的VLLM作为通用CAD任务求解器。2024年，arXiv预印本arXiv:2412.13810。

[22] 鲁格尔·于尔根、迈尔·维尔纳、范哈夫雷·约里克。FreeCAD：开源参数化3D CAD建模软件。2024。

[23] 李学阳、李佳浩、宋宇、楼云忠、周向东。Seek-CAD：一种利用DeepSeek局部推理实现的自优化三维参数化CAD生成建模方法。2025年，arXiv预印本 arXiv:2505.17702。

[24] 袁泽清、兰浩轩、邹强、赵俊波。《3D-PreMise：大型语言模型能否生成具有清晰特征和参数化控制的三维形状？》2024年，arXiv预印本arXiv:2401.06437。

[25] 阿尔拉谢迪·卡梅尔、坦布韦卡尔·普拉迪尤姆纳、扎伊迪·祖尔菲卡尔、兰格瓦瑟·梅根、徐伟、贡博莱·马修。利用视觉-语言模型生成三维设计的CAD代码。2024年，arXiv预印本arXiv:2410.05340。

[26] 罗杰·梅特兰、伯恩哈德·耶根斯坦、伊桑·鲁克、小莫布利、乔杰因·斯诺耶安德烈亚兹·费利克斯·哈伯勒、鲁德·菲施曼、阿米·麦克马伦、杰森·S·德沃拉克、罗曼·德沃拉克西蒙·克莱门茨、博格丹·TheGeek、Spectre5、达利博尔·弗里瓦尔德斯基、达尼埃莱·多拉齐奥乔治、霍伊尤伊、OpenVMP、耶科尔、亚历山大·斯特普克、Mayhem64、卢兹帕兹诺布克、维克托·波贡、斯洛伯林甘特、阿尔诺·博世、巴纳比·沃尔特斯、马蒂·艾登Gumyr/build123d：v0.9.1. 2025

[27] 姚顺宇、赵杰弗里、于迪安、杜楠、沙夫兰·伊扎克，纳拉辛汉、卡蒂克·R、曹源。ReAct：语言模型中推理与行动的协同作用。收录于：第十一届学习表征国际会议。2023年。

[28] 科洛达日尼·马克西姆、塔拉索夫·丹尼斯、热姆丘兹尼科夫·德米特里、尼库林·亚历山大、齐斯曼·伊利亚、沃龙佐娃·安娜、科努申·安东、库连科夫·弗拉基斯拉夫、鲁霍维奇·达尼拉。论文标题：基于在线强化学习的多模态CAD重建。2025年，arXiv预印本arXiv:2505.22914。